

Univerzita Jana Evangelisty Purkyně
V Ústí nad Labem
Přírodovědecká fakulta



Využití vícesnímkového zpracování při rozpoznávání textu s využitím OpenCV a LLM

DIPLOMOVÁ PRÁCE

Vypracovala: Mgr. Bc. Vlasta Michalcová

Vedoucí práce: Mgr. Jiří Fišer, Ph.D.

Studijní program: Aplikovaná informatika

ÚSTÍ NAD LABEM 2026

UNIVERZITA JANA EVANGELISTY PURKYNĚ V ÚSTÍ NAD LABEM

Přírodovědecká fakulta

Akademický rok: 2025/2026

ZADÁNÍ DIPLOMOVÉ PRÁCE

Jméno a příjmení: **Bc. Vlasta MICHALCOVÁ**
Osobní číslo: **F24503**
Studijní program: **N0613P140003 Aplikovaná informatika**
Téma práce: **Využití vícesnímkového zpracování při rozpoznávání textu s využitím OpenCV a LLM**
Zadávající katedra: **Katedra informatiky**

Zásady pro vypracování

Diplomová práce navazuje na bakalářskou práci, v níž byla představena mobilní aplikace CheckChecker využívající OCR a LLM ke skenování účetních dokladů a automatické kategorizaci výdajů. Z analýzy existujících dat je evidentní, že současná implementace je limitována kvalitou pořízeného snímku; zejména u dlouhých účtenek se projevuje nízká kvalita výstupu. Jedním z možných řešení je využití zpracování vícesnímkových dat z videostreamu.

Cílem diplomové práce je návrh a implementace rozšíření aplikace umožňující zpracování účtenek z videostreamu. Uživatel natočí účtenku kamerou a systém automaticky detekuje kvalitní snímky vhodné pro zpracování. Díky vyššímu počtu detailních snímků dojde ke zlepšení výstupu celého procesu.

Mezi základní teoretická východiska patří vícesnímkové zpracování obrazu, hodnocení kvality snímku, OCR a metody využití velkých jazykových modelů (LLM) pro validaci a doplnění údajů a volba vhodných technologií (předpokládá se využití knihovny OpenCV, služby Azure Document Intelligence a nástrojů LLM). Dále je zahrnut postup anonymizace a právní základ zpracování, včetně popisu nakládání s daty v cloudu (lokalita, šifrování, retenční doba).

Součástí řešení bude implementace následujících funkcí:

1. detekce optimálních snímků z videostreamu na základě např. ostroty, kontrastu a stability,
2. automatické pořízení snímku při dosažení dostatečné kvality ("auto-capture"),
3. spojení více snímků do jednoho dokumentu (OCR stitching) pro dlouhé účtenky,
4. OCR zpracování s validací a doplněním klíčových polí pomocí LLM (např. obchodník, datum, částky, DPH).

Výstupem je kromě zdokumentované aplikace i statistické vyhodnocení přesnosti, rychlosti a uživatelské úspěšnosti oproti současnému řešení. Testování proběhne na anonymizovaném vzorku cca 500 účtenek v párovém režimu (jednosnímkový vs. vícesnímkový přístup) se stratifikací podle délky a kvality snímku. Vyhodnocení zahrne CER/WER a exact-match pro celý záznam, precision/recall/F1 pro obchodníka, datum, celkovou částku a DPH, metriky latence a počet manuálních korekcí na dokument a vybrané obrazové metriky z OpenCV. V neposlední řadě je součástí práce zohlednění ekonomických a provozních aspektů včetně modelu nákladů před a po nasazení, měření end-to-end latence a spotřeby zdrojů na zařízení.

Osnova:

Teoretická část

1. OpenCV a metody vícesnímkového zpracování obrazu
2. OCR a Azure Document Intelligence
3. využití LLM pro strukturovanou extrakci a validaci dat
4. ochrana osobních údajů a právní základ zpracování
5. přehled použitých metrik a metod statistického vyhodnocení dat

Praktická část

1. návrh architektury vícesnímkové pipeline
2. návrh metrik pro hodnocení kvality snímku a automatické spouště
3. integrace OpenCV, OCR a LLM modulů do aplikace CheckChecker
4. implementace uživatelského rozhraní s okamžitou zpětnou vazbou
5. testování (experimentální porovnání s jednosnímkovým přístupem)
6. statistické vyhodnocení přesnosti a efektivity
7. zhodnocení ekonomických a provozních aspektů

Forma zpracování diplomové práce: **tištěná/elektronická**

Seznam doporučené literatury:

1. Bradski, Gary; Kaehler, Adrian. *Learning OpenCV 3*. O'Reilly Media, 2017. ISBN 978-1-491-93799-0.
2. Kim, Geewook; Hong, Teakgyu; Yim, Moonbin; Nam, JeongYeon; Park, Jinyoung; Yim, Jinyeong; Hwang, Wonseok; Yun, Sangdoo; Han, Dongyoon; Park, Seunghyun. OCR-Free Document Understanding Transformer (Donut). In: *Computer Vision – ECCV 2022: Proceedings, Part XXVIII*. Springer, 2022, s. 498–517. ISBN 978-3-031-19815-1. DOI: 10.1007/978-3-031-19815-1_29. Dostupné z: https://doi.org/10.1007/978-3-031-19815-1_29
3. Microsoft. *Azure AI Document Intelligence – Developer Guide* [online]. Microsoft, 2025. Dostupné z: <https://learn.microsoft.com/en-us/azure/ai-services/document-intelligence/?view=doc-intel-4.0.0> [cit. 2025-10-23].
4. OpenCV.org. *OpenCV Documentation: Feature Detection, Image Stitching and Camera Calibration* [online]. Verze 4.x, 2024. Dostupné z: <https://docs.opencv.org/4.x/> [cit. 2025-10-23].
5. Szeliski, Richard. *Computer Vision: Algorithms and Applications*. 2. vyd. Springer, 2022. ISBN 978-3-030-34371-2.
6. Hebák, Petr, a kolektiv. *Statistické myšlení a nástroje analýzy dat*. 2. vyd. Praha: Informatorium, 2015. ISBN 978-80-7333-118-4.

Vedoucí diplomové práce: **Mgr. Jiří Fišer, Ph.D.**
Katedra informatiky

Datum zadání diplomové práce: **10. listopadu 2025**

Termín odevzdání diplomové práce: **7. května 2026**

L.S.

doc. RNDr. Michal Varady, Ph.D.
děkan

RNDr. Jiří Škvor, Ph.D.
vedoucí katedry

Prohlášení

Prohlašuji, že jsem tuto diplomovou práci vypracovala samostatně a použila jen pramenů, které cituji a uvádím v přiloženém seznamu literatury.

Byla jsem seznámena s tím, že se na moji práci vztahují práva a povinnosti vyplývající ze zákona c. 121/2000 Sb., ve znění zákona c. 81/2005 Sb., autorský zákon, zejména se skutečností, že Univerzita Jana Evangelisty Purkyně v Ústí nad Labem má právo na uzavření licenční smlouvy o užití této práce jako školního díla podle § 60 odst. 1 autorského zákona, a s tím, že pokud dojde k užití této práce mnou nebo bude poskytnuta licence o užití jinému subjektu, je Univerzita Jana Evangelisty Purkyně v Ústí nad Labem oprávněna ode mne požadovat přiměřený příspěvek na úhradu nákladu, které na vytvoření díla vynaložila, a to podle okolností až do jejich skutečné výše.

V Praze dne 5. května 2026

Podpis: Mgr. Bc. Vlasta Michalcová

Michalcová

Ráda bych poděkovala panu Mgr. Jiřímu Fišerovi, PhD. za opětovné vedení závěrečné práce, za podněty a podporu při studiu.

Za spolupráci při studiu děkuji Martině Kukačkové.

V neposlední řadě děkuji svému manželovi Pavlovi, který při mně stál po celou dobu mého studia a k závěrečné práci a implementaci projektu mi dal cenné připomínky.

VYUŽITÍ VÍCESNÍMKOVÉHO ZPRACOVÁNÍ PŘI ROZPOZNÁVÁNÍ TEXTU S VYUŽITÍM OPENCV A LLM

Abstrakt:

Tato diplomová práce se věnuje návrhu, implementaci a experimentálnímu ověření metody vícesnímkového zpracování obrazu pro zlepšení přesnosti OCR rozpoznávání účtenek v mobilní aplikaci *CheckChecker*. Práce navazuje na bakalářský projekt a rozšiřuje jej o pokročilé metody počítačového vidění s využitím knihovny *OpenCV*. V teoretické části jsou analyzovány principy vícesnímkového zpracování, metriky kvality obrazu, algoritmy pro detekci a párování příznaků, techniky registrace snímků pomocí homografických transformací a skládání dokumentů, a cloudová služba *Azure Document Intelligence*. Praktická část představuje návrh a implementaci vícesnímkové pipeline zahrnující systém hodnocení kvality snímků v reálném čase na základě sady objektivních metrik (ostrost, kontrast, odlesky, sklon, pohyb), algoritmus segmentace účtenky využívající jasovou analýzu s Otsu prahováním, režim automatického skenování s detekcí pohybu kamery pomocí fázové korelace a vlastní pipeline pro skládání více snímků do jednoho kompletního obrazu dokumentu, využívající algoritmus SIFT pro detekci příznaků, párování pomocí RANSAC a *blending* v oblasti švu. Experimentální vyhodnocení na datové sadě 500 účtenek prokázalo statisticky významné zlepšení přesnosti OCR.

Klíčová slova: OCR, vícesnímkové zpracování, OpenCV, image stitching, mobilní aplikace, React Native, Android

UTILIZATION OF MULTI-FRAME PROCESSING FOR TEXT RECOGNITION USING OPENCV AND LLM

Abstract:

This master's thesis focuses on the design, implementation, and experimental evaluation of a multi-frame image processing method for improving OCR accuracy in receipt recognition within the *CheckChecker* mobile application. The work builds upon a previous bachelor's project and extends it with advanced computer vision techniques using the *OpenCV* library. The theoretical part analyses the principles of multi-frame image processing, image quality metrics, feature detection and matching algorithms, image registration using homographic transformations and document stitching, and the *Azure Document Intelligence* cloud service. The practical part presents the design and implementation of a multi-frame pipeline comprising a real-time image quality assessment system based on a set of objective metrics (sharpness, contrast, glare, skew, motion), a receipt segmentation algorithm using brightness analysis with Otsu thresholding, an automatic scanning mode with camera motion detection via phase correlation, and a custom pipeline for compositing multiple frames into a single complete document image, employing the SIFT algorithm for feature detection, RANSAC-based matching, and seam blending. Experimental evaluation on a dataset of 500 receipts demonstrated statistically significant improvements in OCR accuracy.

Keywords: OCR, multi-frame processing, OpenCV, image stitching, mobile application, React Native, Android

Obsah

Úvod.....	13
1. Současný stav aplikace a jeho nedostatky	15
2. Teoretická východiska	21
2.1 Vícesnímkové zpracování obrazu	21
2.2 Metriky kvality obrazu.....	21
2.3 Registrace snímků a geometrické transformace	22
2.4 Skládání snímků (Image Stitching).....	23
2.5 OCR a Azure Document Intelligence	23
2.6 Využití LLM pro kategorizaci dat	26
2.7 Ochrana osobních údajů.....	27
2.8 Metriky a statistické metody	27
3. Popis možných technologií a jejich omezení.....	29
3.1 Knihovny pro zpracování obrazu na mobilních zařízeních	29
3.2 Algoritmy pro detekci příznaků	29
3.3 Přístupy ke skládání snímků	30
3.4 OCR služby	31
3.5 Modely pro kategorizaci pomocí LLM.....	32
4. Návrh řešení.....	33
4.1 Volba knihovny pro zpracování obrazu	33
4.2 Volba algoritmu pro detekci příznaků	33
4.3 Volba přístupu ke skládání snímků.....	34
4.4 Volba OCR služby	34
4.5 Volba modelu pro kategorizaci.....	34
4.6 Architektura navrhované pipeline.....	35
5. Implementace řešení	38
5.1 Architektura aplikace a změněné soubory	39
5.2 Nastavení prahových hodnot quality gates	39
5.3 Implementace ReceiptSegmenter.....	42
5.4 Implementace detekce pohybu a řízení snímání	44
5.5 Implementace stitching pipeline	44
5.6 Změny uživatelského rozhraní.....	45
5.7 Integrace a fallback strategie	47
6. Testování.....	49

6.1 Podmínky testování a standardizace	49
6.2 Datová sada	49
6.3 Vytvoření ground truth	50
6.4 Baseline přístup.....	51
6.5 Statistické vyhodnocení	51
6.6 Výsledky	52
6.7 Velikost efektu	53
6.8 Statistické vyhodnocení	57
Závěr	62
Přílohy.....	64
Seznam použitých zdrojů	65

Úvod

Zájem o osobní finance a jejich evidenci mě provázel již od začátku studia. Když jsem v rámci bakalářské práce [21] vyvíjela mobilní aplikaci *CheckChecker*¹ pro kategorizovanou evidenci výdajů pomocí fotografií účtenek, šlo mi především o to, aby správa financí přestala být nepříjemnou povinností a stala se přirozenou součástí každodenního života. Aplikace využívá cloudovou službu *Azure Document Intelligence* pro OCR (*Optical Character Recognition*, optické rozpoznávání znaků) rozpoznávání textu a *Azure OpenAI* pro (mimo jiné) automatickou kategorizaci položek do předem definovaných kategorií výdajů.

Téměř dvouletý provoz aplikace přinesl kromě zpětné vazby od uživatelů také střízlivý pohled na její slabá místa. Ukázalo se, že největší překážkou spolehlivého fungování není samotný algoritmus kategorizace ani kvalita OCR modelu, ale vstupní snímek, tj. fotografie pořízená uživatelem v reálných, často suboptimálních podmínkách. Rozmazání způsobené třesem ruky, odlesky na lesklém termopapíru, perspektivní zkreslení nebo prostá skutečnost, že dlouhá účtenka se do jednoho záběru nevejde. To vše jsou problémy, které žádný sebelepší OCR systém nedokáže spolehlivě kompenzovat.

Cílem této diplomové práce je navrhnout, implementovat a experimentálně ověřit metodu vícesnímkového zpracování obrazu, která tyto limitace překoná. Vícesnímkový přístup využívá redundantní informace obsažené v sekvenci snímků téže scény: umožňuje automatický výběr nejkvalitnějšího snímku z video sekvence, poskytuje uživateli okamžitou zpětnou vazbu o kvalitě záběru ještě před odesláním na server a umožňuje skládání více snímků pro zachycení dokumentů přesahujících zorné pole kamery.

Práce je členěna do šesti kapitol. První kapitola analyzuje současný stav aplikace a identifikuje konkrétní limitace jednosnímkového přístupu podložené provozními daty. Druhá kapitola shrnuje teoretická východiska – principy vícesnímkového zpracování, metriky kvality obrazu, techniky registrace a skládání snímků, principy OCR a využití LLM (*Large Language Model*, velký jazykový model), právní rámec GDPR a statistické metody použité při vyhodnocení. Třetí kapitola srovnává dostupné technologie a jejich omezení. Čtvrtá kapitola popisuje návrh řešení a zdůvodnění zvolených technologií. Pátá

¹ Dostupná na [Google Play Store](#) a na [App Store](#).

kapitola se věnuje konkrétní implementaci v jazyce Kotlin s využitím knihovny *OpenCV*. Šestá kapitola představuje experimentální vyhodnocení na datové sadě 500 účtenek a diskutuje dosažené výsledky.

1. Současný stav aplikace a jeho nedostatky

Mobilní aplikace pro správu osobních financí představují dynamicky se rozvíjející segment trhu, který reaguje na rostoucí potřebu uživatelů mít přehled o svých výdajích. Klíčovou funkcionalitou těchto aplikací je automatické zpracování účtenek pomocí OCR, které eliminuje nutnost manuálního zadávání dat a výrazně zjednodušuje proces evidence výdajů.

Tato diplomová práce navazuje na bakalářskou práci [\[21\]](#), v rámci které byla vyvinuta mobilní aplikace *CheckChecker* pro kategorizovanou evidenci osobních výdajů. Aplikace využívá cloudovou službu *Azure Document Intelligence* pro OCR rozpoznávání textu z fotografií účtenek a LLM pro automatickou kategorizaci extrahovaných položek do předem definovaných kategorií výdajů. Problematika OCR rozpoznávání, využití LLM pro kategorizaci i architektura původní aplikace byly podrobně popsány v bakalářské práci [\[21\]](#); tato diplomová práce na uvedené základy navazuje a rozšiřuje je o metody vícesnímkového zpracování obrazu.

Během téměř dvouletého provozu aplikace byly shromážděny cenné poznatky o jejím reálném použití a byly identifikovány oblasti vyžadující zlepšení. Provozní zkušenosti odhalily významné limitace jednosnímkového přístupu ke zpracování účtenek. Uživatelé často pořizují snímky v suboptimálních podmínkách – rozmazané fotografie způsobené třesem ruky, odlesky na lesklém termopapíru, neúplné záběry dlouhých účtenek přesahujících zorné pole kamery a perspektivní zkreslení při šikmém snímání jsou faktory, které negativně ovlivňují přesnost OCR rozpoznávání a vedou k nutnosti manuálních oprav, což snižuje uživatelský komfort a efektivitu celého řešení.

Jednosnímkový přístup má tři hlavní strukturální nedostatky:

1. systém je plně závislý na kvalitě jediného snímku a uživatel dostane zpětnou vazbu o jeho nevhodnosti až po odeslání na server a zpracování, nikoli v okamžiku pořizování fotografie. Následně pak dojde k různě závažnému selhání celé OCR pipeline a výsledek digitalizované účtenky je nepoužitelný, viz:



Obrázek 1: Příklad rozmazané účtenky

←

Receipt Details

Failed to read data from the receipt. Please edit manually.

Date: 05/04/2026

Amount (Sum): ? ?

Pair

No data available

Receipt Discounts (0)

This receipt does not contain any discounts.

Receipt Items (0)

Receipt order

Group by category

Sort by price

No items available

|||

○

<

Obrázek 2: Výsledek po zpracování takto rozmazané účtenky

2. při snímání dlouhých účtenek je uživatel nucen volit mezi dvěma nepříjemnými kompromisy: buď se vzdálí natolik, aby zachytil celý doklad, čímž ale sníží efektivní rozlišení textu pod úroveň spolehlivého OCR rozpoznávání, nebo pořídí snímek v dostatečném přiblížení, ale zachytí pouze část účtenky. Pokud se uživatel pokusí o vyfocení dlouhé účtenky jako celku, výsledek OCR (a tedy i kategorizace) bude velmi nepřesný, viz:



Obrázek 3: Dlouhou účtenku je těžké detekovat - a především text ztrácí s oddálením čitelnost

←

PENZION L WANEC

↓

✎

🗑

Pair

PENZION L WANEC

Date: 08/31/2025

Amount (Sum): 212.00 EUR

Amount in default currency: 5187.64

Subtotal (excl. tax & discount): 212.00 EUR

Total tax: 16.00 EUR

Price Including Tax: 212.00 EUR

Receipt Items (4)

+

Receipt order

Group by category

Sort by price

📷

K54

(Restaurants)

✎

🗑

1 ks × 192.00 EUR

Total price: 192.00 EUR

Tax: 16.00 EUR (8.00%)

📷

WOHO

(Restaurants)

✎

🗑

1 ks × 20.00 EUR

Total price: 20.00 EUR

📷

Restaurants

✎

🗑

Restaurants

📷

Restaurants

✎

🗑

Restaurants

Obrázek 4: Výsledné zpracování takové účtenky v jednosnímkovém režimu bylo velmi nepřesné a docházelo ke značné ztrátě informací

- funkce pro nalezení hranic účtenky vyžaduje, aby byly v záběru viditelné všechny čtyři rohy dokumentu, což při částečných záběrech selhává. Tento požadavek je nutný pro správné přečtení všech nezbytných údajů, které účtenka musí mít (typicky v horní části jsou informace o obchodu a obchodníkovi, ve spodní části o celkové částce, měně a datu vystavení účtenky).

Analýza dat z provozu aplikace tyto problémy kvantifikuje: přibližně 23 % účtenek vyžaduje alespoň jednu manuální opravu extrahovaných údajů. U dlouhých účtenek (více než 15 položek) tento podíl stoupá až na 41 %, přičemž nejčastějším problémem je neúplná extrakce položek způsobená tím, že se celá účtenka nevejde do jednoho záběru.

Cílem diplomové práce je navrhnout, implementovat a experimentálně ověřit metodu vícesnímkového zpracování obrazu, která tyto limitace překoná. Vícesnímkový přístup využívá redundantní informace obsažené v sekvenci snímků téže scény. Umožňuje automatický výběr nejostřejšího snímku z video sekvence, poskytuje uživateli okamžitou zpětnou vazbu o kvalitě aktuálního záběru a umožňuje skládání více snímků pro zachycení dlouhých dokumentů, které se nevejdou do jednoho záběru.

2. Teoretická východiska

2.1 Vícesnímkové zpracování obrazu

Základním principem vícesnímkového zpracování je využití časové redundance, tedy skutečnosti, že po sobě jdoucí snímky video sekvence obsahují vysoce korelovanou informaci o téže scéně. Tato korelace vzniká proto, že mezi jednotlivými snímky (při typické snímkové frekvenci 30 fps je interval přibližně 33 ms) se scéna mění jen minimálně.

Klíčovým poznatkem je, že zatímco užitečný signál (obsah scény, tedy text na úctence) zůstává mezi snímky konzistentní, šum je nezávislý a náhodný. Při průměrování většího počtu správně zarovnaných snímků se užitečný signál zachovává, zatímco náhodný šum se vzájemně vyruší. Konkrétně platí, že při průměrování N snímků se směrodatná odchylka šumu snižuje faktorem $\sqrt{N}$, což vede ke zlepšení poměru signálu k šumu (SNR). [\[13\]](#)

V literatuře jsou rozlišovány dva základní přístupy k využití časové redundance. První přístup, tzv. *multi-frame averaging*, spočívá v zarovnání a průměrování několika snímků téže scény za účelem redukce šumu. Tento přístup je účinný především pro zlepšení SNR u statických scén s nízkým osvětlením, vyžaduje však subpixelově přesnou registraci, která je výpočetně náročná. [\[13\]](#) Druhý přístup spočívá v automatickém výběru nejvyššího snímku z video sekvence na základě objektivních metrik kvality. [\[29\]](#)

2.2 Metriky kvality obrazu

Pro automatický výběr optimálního snímku z video sekvence je nezbytné definovat objektivní metriky kvality, které korelují s úspěšností OCR rozpoznávání. Přehled metod hodnocení kvality obrazu dokumentů podávají například Alaei et al. [\[1\]](#).

Variance of Laplacian (VoL) je standardní metrikou pro měření ostrosti obrazu. Laplaceův operátor je diferenciální operátor druhého řádu, který zvýrazňuje oblasti s rychlou změnou intenzity, tedy hrany a jemné detaily. Rozptyl hodnot Laplaceova operátoru indikuje, kolik hran obraz obsahuje – ostrý obraz má vysoký rozptyl, zatímco rozmazaný obraz má rozptyl nízký. [\[13\]](#) Alternativními metrikami ostrosti jsou *gradient magnitude* nebo *Tenengrad*, které měří sílu hran v obraze. [\[23\]](#)

Histogram Spread je metrika pro hodnocení kontrastu a expozice snímku. Sleduje podíl pixelů v tmavém konci histogramu (*lowTailPct*) a světlém konci histogramu (*highTailPct*). [13] Podexponované snímky ztrácejí detail v tmavých oblastech, zatímco přexponované snímky ztrácejí detail ve světlých oblastech.

Detekce odlesků (*Glare Detection*) kvantifikuje podíl přesvětlených oblastí způsobených odrazem světla od povrchu dokumentu. Na rozdíl od celkové přexpozice měřené histogramem se odlesky typicky vyskytují v lokalizovaných oblastech. Pro jejich detekci se obvykle využívá prahování s vysokou hodnotou intenzity a morfologická operace otevření pro odstranění izolovaných pixelů.

Skew Angle měří úhel natočení dokumentu vůči horizontální ose. Pro výpočet se využívají detekované rohy dokumentu a měří se odchylka horní a dolní hrany od horizontály. [13]

Motion měří pohyb mezi po sobě jdoucími snímky. Tato metrika je klíčová pro prevenci *motion blur* – i krátký pohyb kamery během expozice může způsobit rozmazání textu a výrazně snížit čitelnost znaků. [13]

2.3 Registrace snímků a geometrické transformace

Registrace (zarovnání) snímků je proces nalezení geometrické transformace, která mapuje body jednoho snímku na odpovídající body v druhém snímku. [29] Jedná se o nezbytný krok pro skládání více snímků téhož dokumentu do jednoho kompletního obrazu.

Homografie je vhodným modelem transformace pro planární scény. Jedná se o projektivní transformaci popsanou maticí 3×3 s osmi stupni volnosti (devátý prvek je normalizován na 1), která mapuje body z jedné roviny do druhé. [29] Pro dokumenty, které jsou planárními (rovinnými) objekty, homografie přesně popisuje vztah mezi dvěma pohledy na tutéž scénu z různých úhlů nebo pozic.

Pro detekci a popis příznaků sloužících k výpočtu homografie existuje několik etablovaných algoritmů. SIFT (*Scale-Invariant Feature Transform*) [18] detekuje klíčové body invariantní vůči změnám měřítka a rotace a popisuje je deskriptorem robustním vůči změnám osvětlení. ORB (*Oriented FAST and Rotated BRIEF*) [26] představuje výpočetně

méně náročnou alternativu, která nevyžaduje licenci. SURF (*Speeded Up Robust Features*) [8] nabízí kompromis mezi rychlostí a robustností. Každý z těchto detektorů má odlišné vlastnosti z hlediska přesnosti, výpočetní náročnosti a licenčních podmínek.

Algoritmus RANSAC (*Random Sample Consensus*) [12] slouží pro robustní odhad homografie z párů příznaků. RANSAC iterativně náhodně vybírá minimální počet bodů potřebných pro výpočet modelu (4 body pro homografii), vypočítá model a spočítá počet inlierů (datových bodů odpovídajících odhadovanému modelu v rámci stanovené prahové hodnoty odchylky). Po stanoveném počtu iterací je vybrán model s největším počtem inlierů. Tento přístup je robustní vůči outlierům, tedy chybným párům příznaků, které by při použití metody nejmenších čtverců vedly k nesprávnému odhadu transformace.

2.4 Skládání snímků (*Image Stitching*)

Pro dokumenty delší než zorné pole kamery je nutné kombinovat více částečných snímků do jednoho kompletního obrazu. V literatuře jsou popisovány dva základní přístupy. [10]

První přístup využívá hotová řešení pro panoramatické skládání snímků, jako je například *OpenCV Stitcher API* [30]. Tato API jsou optimalizována pro fotografická panoramata a automaticky řeší detekci příznaků, párování, výpočet homografií i *blending* (prolínání obrazů v oblasti překryvu). Předpokládají však, že snímky zachycují scénu z různých úhlů pohledu se společným středem projekce, což neodpovídá všem případům užití.

Druhý přístup spočívá v implementaci vlastní pipeline² přizpůsobené konkrétnímu typu dokumentu a způsobu snímání. Taková pipeline typicky zahrnuje detekci příznaků v překrývajících se oblastech sousedních snímků, jejich párování, výpočet transformace a následné *blending*. *Blending* lze realizovat váženým průměrováním s lineárním přechodem vah (*alpha blending*) [10], přičemž optimální pozice švu se hledá minimalizací rozdílu intenzit mezi snímky v překrývajících se oblastech.

2.5 OCR a *Azure Document Intelligence*

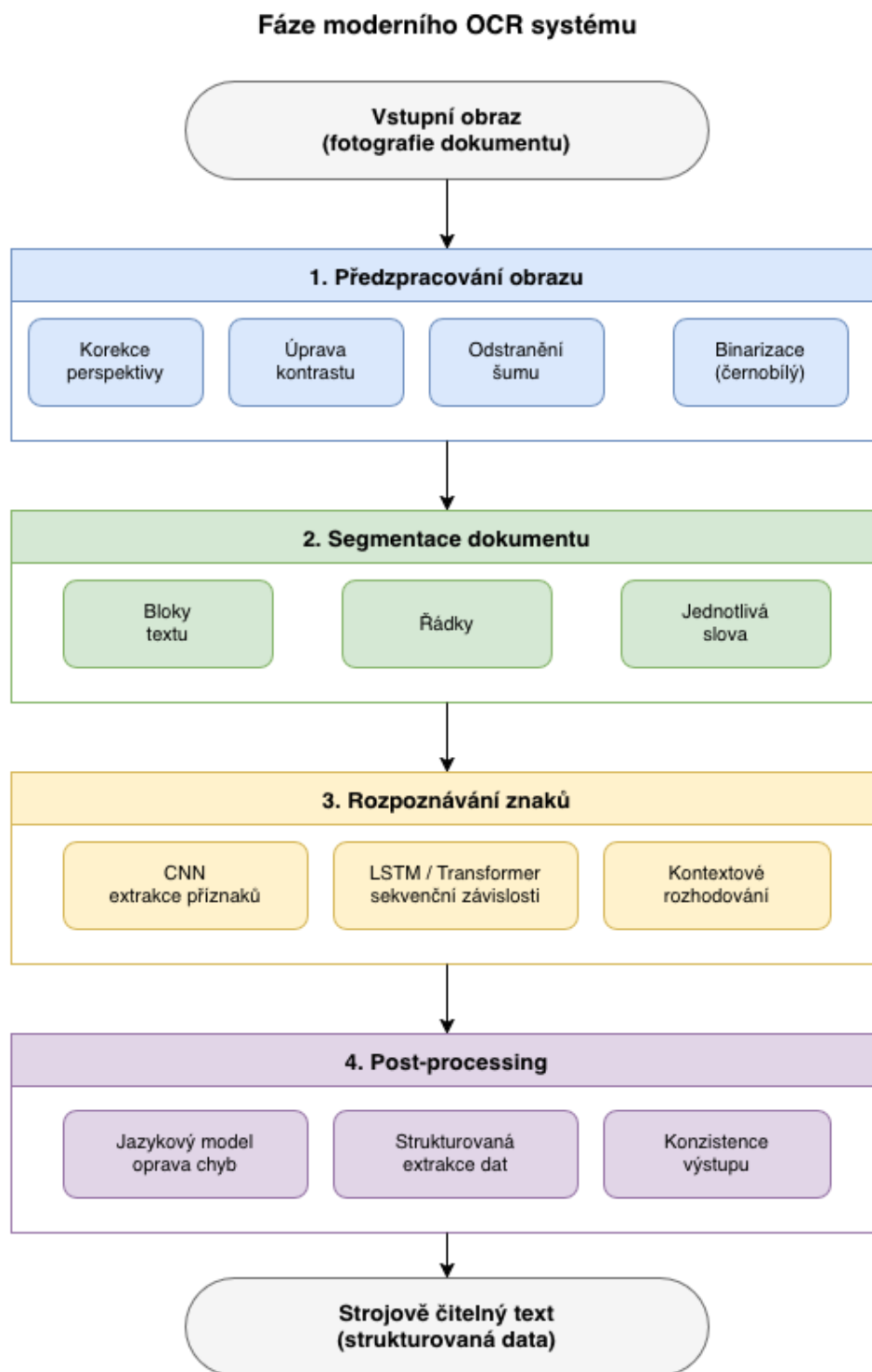
² *Pipeline* (z angl. potrubí) označuje v informatice řetězec zpracovatelských kroků, kde výstup jednoho kroku tvoří vstup následujícího. V kontextu zpracování obrazu jde o sekvenci operací prováděných nad snímkem od předzpracování přes detekci příznaků až po výslednou transformaci. [9]

Optické rozpoznávání znaků (OCR) je technologie umožňující převod obrazů obsahujících text na strojově čitelný text. Problematika OCR byla podrobně představena v bakalářské práci [21], kde byl popsán historický vývoj technologie. V této kapitole jsou shrnuty pouze aspekty relevantní pro vícesnímkové zpracování.

Moderní OCR systémy typicky pracují v několika fázích. První fází je předzpracování obrazu, které zahrnuje korekci perspektivy, úpravu kontrastu, odstranění šumu a binarizaci³. Následuje segmentace, tedy rozdělení dokumentu na bloky textu, řádky a jednotlivá slova. [13] Jádrem systému je rozpoznávání znaků pomocí neuronových sítí. [14] Poslední fází je *post-processing* využívající jazykové modely pro opravu chyb a zlepšení konzistence výstupu. [14]

Současné OCR systémy používají konvoluční neuronové sítě (CNN) [14] pro extrakci vizuálních příznaků z obrazu a rekurentní neuronové sítě (RNN) architektury LSTM nebo moderní *Transformer* architektury pro modelování sekvenčních závislostí mezi znaky. [14] Přesnost OCR je přitom silně závislá na kvalitě vstupního snímku – klíčovými faktory jsou ostrost, kontrast, přítomnost odlesků, perspektivní zkreslení a efektivní rozlišení textu.

³ Binarizace je proces převodu obrazu do dvouúrovňové (černobílé) reprezentace, při níž je každému pixelu přiřazena hodnota 0 nebo 1 (resp. 0 nebo 255) podle toho, zda jeho intenzita překračuje stanovenou prahovou hodnotu. Výsledný binární obraz usnadňuje následnou segmentaci a detekci objektů.



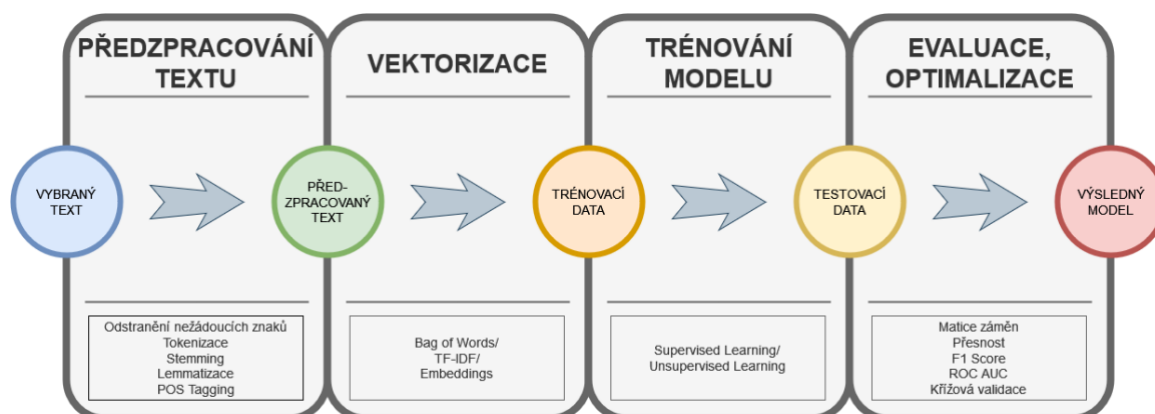
Obrázek 5: Fáze moderního OCR systému

Azure Document Intelligence je cloudová služba společnosti *Microsoft* pro inteligentní zpracování dokumentů. Služba kombinuje pokročilé OCR s extrakcí strukturovaných dat a je optimalizována pro zpracování běžných typů dokumentů včetně účtenek, faktur

a formulářů. [5] Pro účtenky je k dispozici model *prebuilt-receipt*, který automaticky extrahuje metadata účtenky (název obchodníka, adresa, datum, čas), finanční údaje (celková částka, DPH, platební metoda) a jednotlivé položky (název, množství, jednotková cena, celková cena). Model podporuje více jazyků včetně češtiny. [4] Služba je dostupná prostřednictvím REST API a zpracování probíhá asynchronně. [3]

2.6 Využití LLM pro kategorizaci dat

Velké jazykové modely (LLM) jsou neuronové sítě s miliardami parametrů trénované na rozsáhlých textových datech. Jejich využití pro kategorizaci a extrakci informací bylo podrobně představeno v bakalářské práci [21]. V kontextu zpracování účtenek lze LLM využít pro automatickou kategorizaci položek do předem definovaných kategorií výdajů na základě jejich textového popisu.



Obrázek 6 převzat z bakalářské práce [21]

Kvalita kategorizace závisí významně na struktuře promptu. Technika *few-shot learning* spočívá v zařazení příkladů správné kategorizace přímo do promptu, čímž model získá konkrétní vzor požadovaného chování bez nutnosti přetrénování. [6] Strukturovaný výstupní formát (například CSV) usnadňuje strojové zpracování odpovědi modelu a snižuje riziko nevalidního výstupu.

Perspektivní rozšíření představují multimodální modely jako *GPT-4 Vision* nebo *Claude 3*, které umožňují kombinovat vizuální informaci s jazykovým kontextem. [2] [7] Takový model by teoreticky mohl přímo zpracovat fotografii účtenky a provést jak OCR, tak kategorizaci v jednom kroku, přičemž by mohl využít vizuální kontext pro lepší zvládnání

nejednoznačných případů. V praxi by to ale implementace nemusela vést k dobrým výsledkům, protože tento nápad kombinuje kroky, které vyžadují zcela jiné přístupy. OCR vyžaduje co nejméně hádání (v ideálním případě input = output). Naopak při kategorizaci je žádoucí, aby si LLM vymýšlelo odpovědi (podle toho se nastavuje teplota pro LLM).

2.7 Ochrana osobních údajů

Zpracování účtenek zahrnuje nakládání s daty, která mohou obsahovat osobní údaje ve smyslu GDPR. [22] Účtenky typicky obsahují údaje o obchodníkovi a transakci; potenciálně citlivější jsou čísla věrnostních karet nebo údaje o platební kartě.⁴

Primárním právním základem pro zpracování těchto dat by mělo být smluvní plnění (čl. 6 odst. 1 písm. b GDPR). Uživatel aktivně nahrává účtenky za účelem jejich evidence a zpracování je nezbytné pro poskytnutí služby. Z hlediska technických opatření by komunikace měla probíhat přes HTTPS s TLS 1.2+, data v databázi by měla být šifrována a přístup chráněn autentizací. Cloudové služby využívané pro zpracování by měly být certifikovány podle relevantních norem a zpracovávat data v regionu EU. Datové sady využívané pro experimentální část by měly být před zařazením anonymizovány – tedy zbaveny osobních údajů, které by umožňovaly identifikaci konkrétních osob.

2.8 Metriky a statistické metody

Pro hodnocení přesnosti OCR jsou standardně používány metriky *Character Error Rate* (CER) a *Word Error Rate* (WER), obě vycházející z Levenshteinovy editační vzdálenosti. [17]

CER měří chybovost na úrovni jednotlivých znaků podle vzorce

$$CER = (S + D + I) / N,$$

⁴ Tato informace se na účtenkách zpravidla nevyskytuje, ale pokud se přesto objeví, aplikace tuto informaci ukládá pouze v omezené formě, například u platební karty jsou uložena pouze poslední 4 čísla, díky čemuž může uživatel identifikovat použitou kartu, ale nelze ji zneužít bez dalších osobních dat.

kde S je počet substitucí, D počet mazání, I počet vložení a N celkový počet znaků v referenčním textu. [17] WER analogicky měří chybovost na úrovni celých slov. WER je vždy vyšší nebo rovna CER, protože jediná chyba ve znaku způsobí chybu celého slova.

Pro hodnocení extrakce strukturovaných entit jsou standardně používány metriky z oblasti *Information Extraction* [20]:

- *Precision* (podíl správně extrahovaných entit z celkového počtu extrahovaných),
- *Recall* (podíl správně extrahovaných entit z celkového počtu entit existujících v dokumentu)
- *F1-score* jako jejich harmonický průměr: $F1 = 2 \times (P \times R) / (P + R)$. [20]

Pro statistické porovnání dvou metod na párových datech jsou vhodné párové testy. Při splnění předpokladu normality rozdílů je vhodný párový t-test; pokud data normalitu nesplňují, používá se neparametrický Wilcoxonův párový test (*signed-rank test*), který je robustní vůči outlierům. [27] [36] Při mnohonásobném testování více hypotéz je nezbytné kontrolovat chybu prvního druhu – standardním přístupem je *Bonferroniho korekce*, která upravuje hladinu významnosti vydělením počtem testovaných hypotéz. [27] Statistická významnost sama o sobě neříká nic o praktickém dopadu rozdílu; pro jeho kvantifikaci slouží Cohenovo d , které vyjadřuje rozdíl středních hodnot v jednotkách směrodatné odchylky. Hodnoty $d \geq 0,2$ indikují malý efekt, $0,2 \leq d < 0,8$ střední efekt a $d \geq 0,8$ velký efekt. [11]

3. Popis možných technologií a jejich omezení

3.1 Knihovny pro zpracování obrazu na mobilních zařízeních

Při implementaci vícesnímkového zpracování na platformě Android přicházelo v úvahu několik knihoven. Každá z nich nabízí odlišný kompromis mezi výkonem, rozsahem funkcí a složitostí integrace.

OpenCV [23] je multiplatformní open-source knihovna počítačového vidění s více než dvěma tisíci optimalizovanými algoritmy pokrývajícími celé spektrum operací – od základního předzpracování obrazu přes detekci příznaků až po geometrické transformace a skládání snímků. Pro platformu Android je dostupná prostřednictvím oficiálního *OpenCV4Android SDK* [35], které zpřístupňuje funkce knihovny z kódu v Kotlinu prostřednictvím *Java Native Interface*⁵. *OpenCV* má rozsáhlou dokumentaci a aktivní komunitu a je multiplatformní – stejný kód lze s minimálními úpravami použít i na iOS.

GPUImage je knihovna primárně určená pro aplikaci filtrů a vizuálních efektů s využitím hardwarové akcelerace GPU. Je optimalizována pro mobilní zařízení a nabízí vysoký výkon pro jednoduché obrazové operace. Nepodporuje však komplexní algoritmy jako detekci příznaků SIFT nebo výpočet homografie, což ji pro potřeby stitchingové pipeline diskvalifikuje.

Nativní *Android Camera2 API* v kombinaci s *RenderScript*⁶ by umožnilo přímý přístup k video streamu s minimální latencí a výpočty na GPU. *RenderScript* je však od verze Android 12 označen jako zastaralý (*deprecated*) a pro nové projekty se nedoporučuje. Implementace složitých algoritmů od základu by navíc výrazně prodloužila dobu vývoje.

3.2 Algoritmy pro detekci příznaků

⁵ Rozhraní umožňující propojit kód běžící na virtuálním stroji Javy s nativními programy a knihovnami napsanými v jiných jazycích – např. C, C++, Assembler, apod., které jsou zkompileované pro určitý hardware, případně operační systém. Jedná se tedy o jakýsi převodní můstek, pomocí kterého se můžeme dostat za hranice virtuálního stroje. <https://www.baeldung.com/jni>

⁶ Alternativa v podobě *GPUImage* byla zamítnuta především proto, že tato knihovna je primárně určena pro aplikaci filtrů a jednodušší transformace, nikoliv pro komplexní operace jako je detekce příznaků SIFT nebo výpočet homografie. Nativní *RenderScript* by sice nabídl lepší výkon pro některé operace, ale vyžadoval by implementaci algoritmů od základu, což by značně prodloužilo dobu vývoje bez jasného přínosu pro kvalitu výsledku.

Pro výpočet homografie mezi dvojicí snímků je nutné nejprve detekovat a spárovat odpovídající body – příznaky – v obou snímcích. V literatuře jsou pro tento účel zavedeny tři hlavní algoritmy. [29]

SIFT (*Scale-Invariant Feature Transform*) [18] detekuje klíčové body invariantní vůči změnám měřítka a rotace a popisuje je 128dimenzionálním deskriptorem robustním vůči změnám osvětlení. SIFT je výpočetně nejnáročnější z uvedených alternativ, poskytuje však nejrobustnější výsledky i při větších změnách měřítka, rotace a osvětlení mezi snímky. Od roku 2020 vypršela patentová ochrana algoritmu, takže jej lze volně využívat i v komerčních aplikacích. [19]

ORB (*Oriented FAST and Rotated BRIEF*) [26] je výpočetně výrazně méně náročnou alternativou a nevyžaduje licenci. Při větších změnách měřítka nebo osvětlení však poskytuje méně robustní párování než SIFT, což může vést k chybám při odhadu homografie.

SURF (*Speeded Up Robust Features*) [8] nabízí kompromis mezi rychlostí SIFT a robustností ORB. Algoritmus je patentově chráněný, což jeho použití v komerčních aplikacích bez licence vylučuje.

3.3 Přístupy ke skládání snímků

Pro skládání více částečných snímků dokumentu do jednoho celku existují dva základní přístupy.

OpenCV Stitcher API poskytuje hotové řešení pro panoramatické skládání snímků a automaticky řeší detekci příznaků, párování, výpočet homografií i *blending*. [30] Toto API je optimalizováno pro fotografická panoramata, kde snímky zachycují scénu z různých úhlů pohledu se společným středem projekce. Pro případ vertikálního posouvání kamery nad dokumentem – kdy kamera postupuje rovnoběžně s rovinou dokumentu – tato podmínka neplatí a API produkuje nesprávné výsledky.

Vlastní stitching pipeline přizpůsobená konkrétnímu *use case* tento problém obchází tím, že je navržena přímo pro vertikální skládání dokumentů. Taková pipeline obvykle zahrnuje

detekci příznaků pouze v překrývajících se oblastech sousedních snímků, jejich párování, výpočet homografie pomocí RANSAC a *blending* v oblasti švu. Nevýhodou je vyšší implementační náročnost.

3.4 OCR služby

Pro optické rozpoznávání textu z fotografií účtenek přicházely v úvahu čtyři hlavní technologie.

Tesseract OCR [28] je vyspělý open-source projekt s dlouhou historií, dostupný zdarma. Jeho nasazení na mobilním zařízení vyžaduje významné úsilí při optimalizaci výkonu.⁷

Google Cloud Vision API a *Amazon Textract* jsou cloudové služby s pokročilými OCR modely. Obě v posledních letech zavedly specializované modely pro účtenky – *Google Document AI Receipt Parser* a *Amazon Textract Expense API*. Jejich integrace by však vyžadovala závislost na infrastruktuře mimo ekosystém *Microsoft Azure*, který aplikace *CheckChecker* využívá pro ostatní cloudové služby.

Azure Document Intelligence [5] je cloudová služba kombinující pokročilé OCR s extrakcí strukturovaných dat, optimalizovaná pro běžné typy dokumentů včetně účtenek. Model *prebuilt-receipt* automaticky extrahuje metadata (název obchodníka, datum, čas), finanční údaje (celková částka, DPH) a jednotlivé položky. Model je trénován na rozsáhlé datové sadě z různých zemí a podporuje češtinu. [4] Služba je dostupná prostřednictvím REST API s asynchronním zpracováním. [3] Pokud model *prebuilt-receipt* selže u nestandardního formátu účtenky, lze jako zálohu použít obecnější model *prebuilt-read*, který provede pouze OCR bez strukturované extrakce.⁸

⁷ *Tesseract OCR* jsem zamítla z několika důvodů. Přestože se jedná o vyspělý open-source projekt s dlouhou historií, jeho nasazení na mobilním zařízení by vyžadovalo významné úsilí při optimalizaci výkonu. Navíc *Tesseract* neposkytuje specializované modely pro účtenky – musel by být použit obecný model, jehož přesnost pro strukturované dokumenty jako účtenky je nižší než u specializovaných cloudových služeb.

⁸ Typická latence zpracování účtenky je 2–3 sekundy v závislosti na velikosti dokumentu a aktuálním zatížení služby – hodnota vychází z měření v rámci provozu aplikace *CheckChecker*. Azure dokumentace garantuje pouze variabilitu latence odpovídající multitenant architektuře služby.

3.5 Modely pro kategorizaci pomocí LLM

Pro automatickou kategorizaci položek účtenek byly v průběhu vývoje aplikace zvažovány různé modely velkých jazykových modelů.

GPT-3.5 Turbo byl v době zahájení vývoje aplikace (2023) standardní volbou pro produkční nasazení díky dobrému poměru ceny a výkonu. *OpenAI* jej postupně přesouvá do kategorie legacy modelů s omezenou podporou a bez dalšího vývoje, což snižuje jeho dlouhodobou perspektivu.

GPT-4o-mini je novější model nabízející při srovnatelných nebo nižších nákladech vyšší přesnost kategorizace. Lépe zvládá české texty a specifické názvy produktů vyskytující se na českých účtenkách. Model je dostupný prostřednictvím *Azure OpenAI Service*, což zajišťuje soulad s evropskými požadavky na zpracování dat a stabilní SLA pro produkční provoz. [\[31\]](#) [\[32\]](#)

Multimodální modely jako *GPT-4 Vision* nebo *Claude 3* umožňují kombinovat vizuální informaci s jazykovým kontextem a mohly by teoreticky provést OCR i kategorizaci v jednom kroku. [\[2\]](#) [\[7\]](#) Jejich nasazení je však spojeno s vyšší latencí, vyššími náklady a nejistotou ohledně konzistence výstupů ve srovnání s kombinací specializované OCR služby a textového LLM. Alternativní přístup představují end-to-end modely jako Donut [\[34\]](#), které zpracovávají dokument přímo z obrazu bez předchozího OCR kroku a eliminují tak chyby vzniklé v OCR fázi.

4. Návrh řešení

4.1 Volba knihovny pro zpracování obrazu

Pro implementaci vícesnímkového zpracování jsem zvolila knihovnu *OpenCV* [23]. Hlavním důvodem je širší pokrytí – knihovna poskytuje kompletní sadu funkcí potřebných pro celou pipeline v rámci jediné závislosti: od předzpracování obrazu a výpočtu metrik kvality přes detekci příznaků až po geometrické transformace a skládání snímků. *GPUImage* tuto šíři nenabízí a nativní *Camera2 API* s *RenderScriptem* je od Android 12 označeno jako zastaralé, což by ohrozilo dlouhodobou udržitelnost řešení.

Integrace probíhá prostřednictvím *OpenCV4Android SDK* verze 4.8, které zpřístupňuje funkce knihovny z Kotlinu přes *Java Native Interface*. Vícesnímkové zpracování je implementováno kompletně v nativní vrstvě, protože *real-time* analýza video streamu a výpočetně náročné operace *OpenCV* by v interpretované vrstvě React Native způsobovaly znatelné zpoždění. Respektive nejde ani tak o zpoždění, jako o nedostatečný výkon React Native abstrakce. *JNI* je potřeba, protože potřebujeme zpracovat určitý počet snímků za vteřinu, abychom mohli vybrat vhodný snímek. Pro účely této práce jsou využity především moduly *imgproc* pro předzpracování obrazu, *features2d* pro detekci příznaků SIFT, *calib3d* pro výpočet homografie a *core* pro základní operace s maticemi.

4.2 Volba algoritmu pro detekci příznaků

Pro detekci a párování příznaků při stitchingu jsem zvolila algoritmus SIFT [18]. Klíčovým argumentem je robustnost – stitching se provádí pouze jednou po dokončení celého skenování, nikoliv v reálném čase, takže vyšší výpočetní náročnost SIFT oproti ORB nepředstavuje praktický problém. Naopak robustnost vůči změnám měřítka a osvětlení mezi snímky je pro spolehlivé párování zásadní. SURF by sice nabídl dobrý kompromis, ale jeho patentová ochrana jej pro použití v komerční aplikaci bez licence vylučuje. Od roku 2020 platnost patentu SIFT vypršela [19], takže jej lze volně použít i v produkčním prostředí.

4.3 Volba přístupu ke skládání snímků

Po experimentálním ověření jsem zamítla použití *OpenCV Stitcher API* a rozhodla se implementovat vlastní stitching pipeline. *OpenCV Stitcher API* předpokládá, že snímky zachycují scénu z různých úhlů pohledu se společným středem projekce, což odpovídá fotografickým panoramatům. Při vertikálním posouvání kamery nad účtenkou tato podmínka neplatí – kamera se pohybuje rovnoběžně s rovinou dokumentu – a API v tomto případě produkuje nesprávné výsledky.

Navržená vlastní pipeline je optimalizována pro specifický případ vertikálního skládání dokumentů a pracuje v následujících krocích: detekce SIFT příznaků v překrývajících se oblastech sousedních snímků (dolní třetina horního snímku a horní dvě třetiny dolního snímku⁹), párování příznaků pomocí *BFMatcher* s *Lowe ratio testem*, robustní odhad homografie pomocí RANSAC [12] a *blending* s hledáním optimálního švu. [10]

4.4 Volba OCR služby

V aplikaci *CheckChecker* je od jejího vzniku využívána služba *Azure Document Intelligence*, jejíž volba byla zdůvodněna v bakalářské práci [21]. V rámci diplomové práce jsem se rozhodla v této volbě pokračovat. Hlavním důvodem je kontinuita – aplikace má funkční integraci s *Azure* a přechod na jinou službu by vyžadoval netriviální úpravy backendu bez jasného přínosu pro kvalitu výsledků, protože navrhované vícesnímkové zpracování funguje jako *preprocessing* vrstva nezávislá na volbě OCR služby.

4.5 Volba modelu pro kategorizaci

Na základě srovnání modelů popsaného v kapitole 3.5 jsem zvolila model *GPT-4o-mini* nasazený prostřednictvím *Azure OpenAI Service*. Hlavními důvody byly ukončování podpory *GPT-3.5 Turbo*, vyšší přesnost kategorizace při srovnatelných nákladech a lepší zvládání českých textů. [32]

⁹ Lineární vážený přechod (*linear blending* / *alpha blending*) jako technika kompozice obrazu. [10]

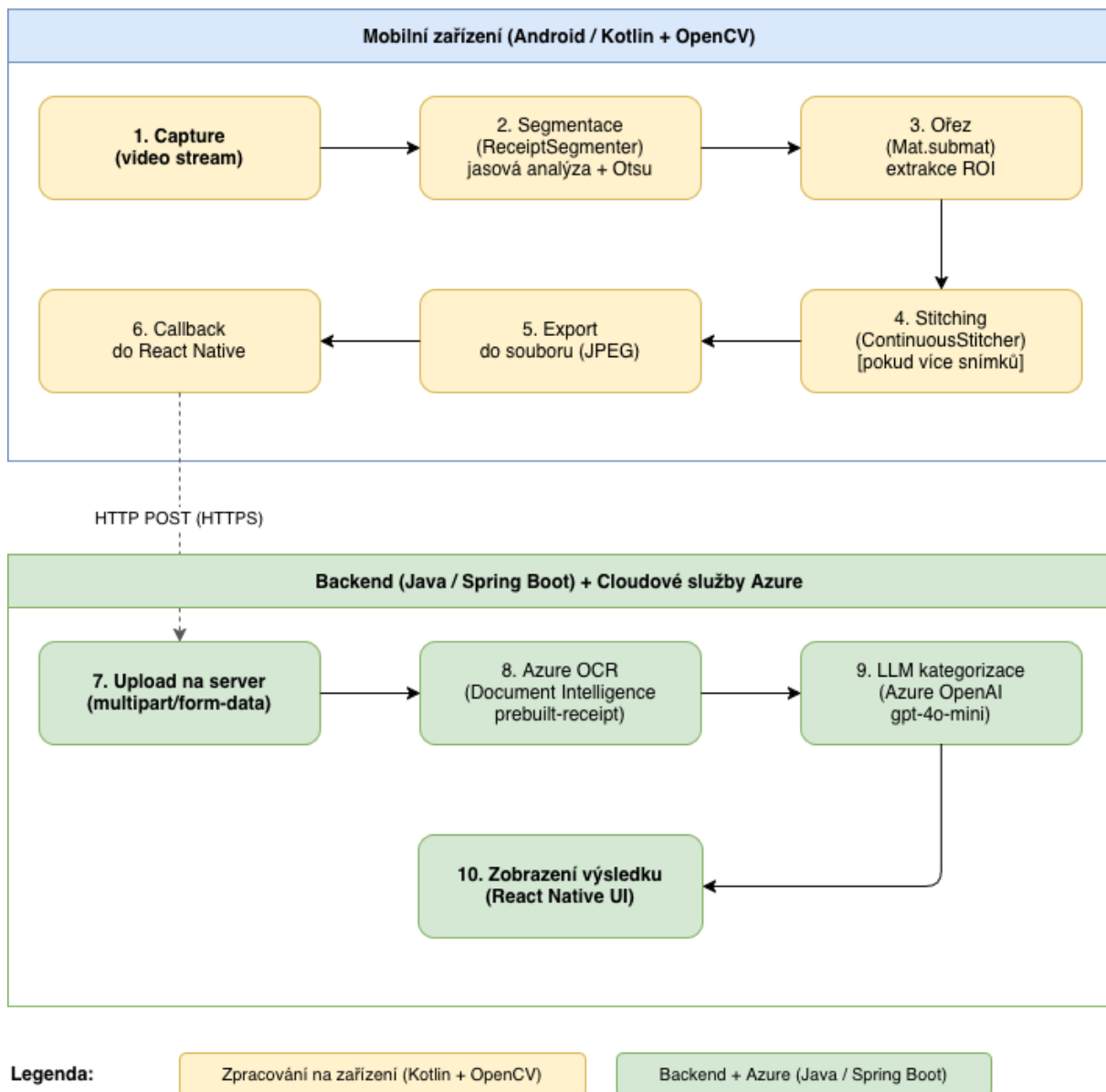
4.6 Architektura navrhované pipeline

Navržené řešení kombinuje dva přístupy reagující na tři hlavní limitace původní implementace identifikované v [kapitole 1](#): závislost na kvalitě jediného snímku bez okamžité zpětné vazby, nemožnost zachytit dlouhé účtenky v plném rozlišení a selhání detekce okrajů při částečných záběrech.

První přístup – automatický výběr kvalitního snímku – řeší první limitaci. Pipeline kontinuálně analyzuje video stream a hodnotí každý snímek sadou metrik kvality (*quality gates*). Zpětná vazba je poskytována uživateli v reálném čase, takže problém je identifikován ještě před odesláním snímku na server.

Druhý přístup – podpora pro snímání dlouhých dokumentů – řeší druhou a třetí limitaci. Pipeline detekuje pohyb kamery pomocí *phase correlation* [\[31\]](#) a automaticky pořizuje dílčí snímky při překročení prahové hodnoty posunu. Výsledné snímky jsou složeny do jednoho kompletního obrazu pomocí vlastní stitching pipeline.

Pipeline zpracování účtenky — CheckChecker

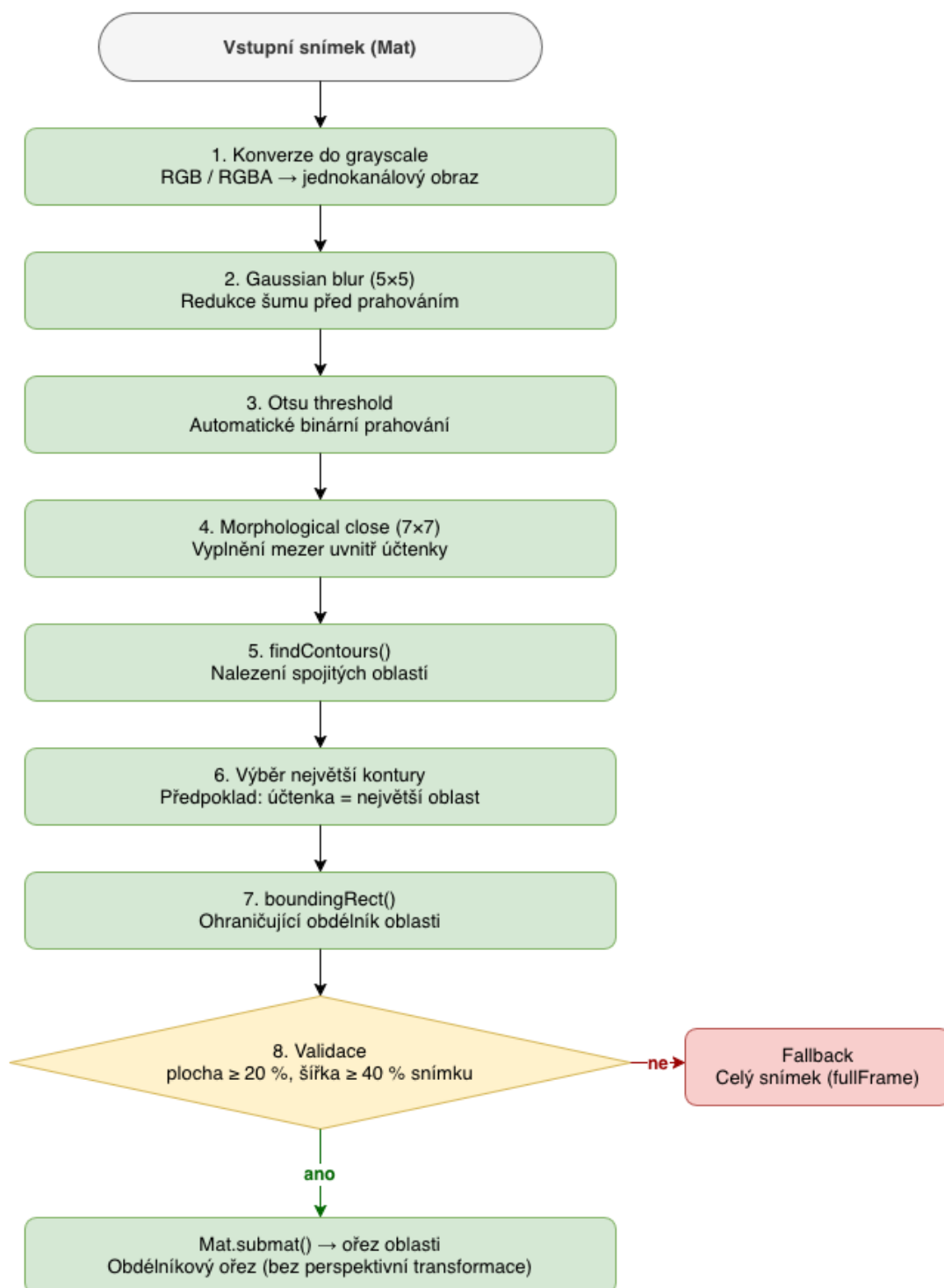


Obrázek 7: Průběh zpracování účtenky

Klíčovými komponentami návrhu jsou:

Receipt Segmenter – nová komponenta pro segmentaci účtenky ze snímku. Na rozdíl od původní funkce hledající čtyřúhelník s viditelností všech čtyř rohů využívá jasovou analýzu s *Otsu* prahováním a dokáže správně segmentovat i částečné záběry, kde část účtenky přesahuje okraje snímku.

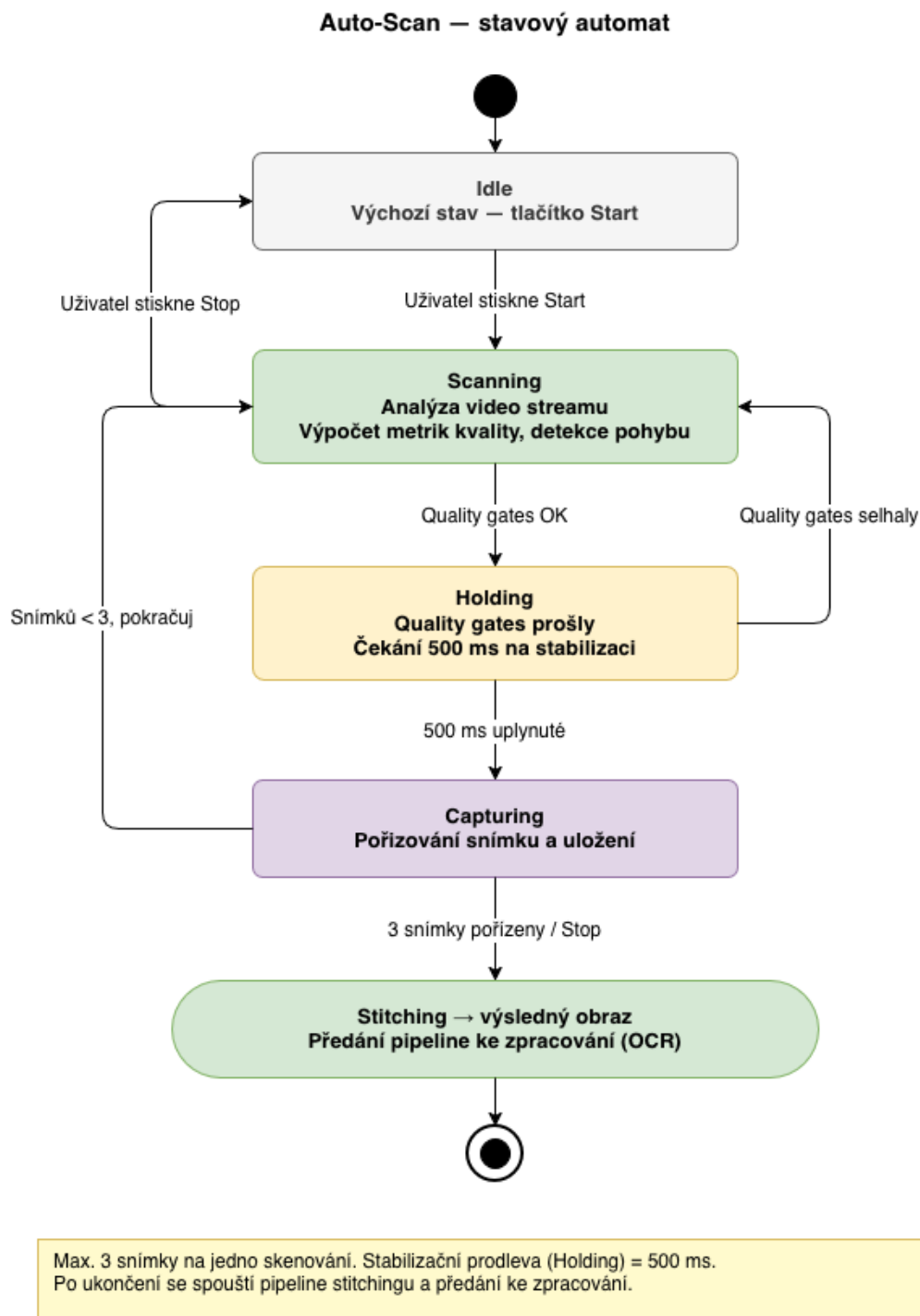
ReceiptSegmenter — flowchart



Pozn.: Algoritmus neprovádí perspektivní warping.
Azure Document Intelligence provádí vlastní interní korekce natočení.

Obrázek 8: Receipt Segmenter

Auto-Scan Mode – režim kontinuálního skenování s automatickým pořizováním snímků při splnění *quality gates* a detekci pohybu kamery. Pro tento režim jsou *quality gates* upraveny oproti standardnímu foto režimu: kontroly vyžadující viditelnost všech čtyř rohů dokumentu jsou vypnuty, protože při částečném záběru kompletní čtyřúhelník neexistuje.



Obrázek 9: Stavový automat komponenty Auto-Scan

5. Implementace řešení

5.1 Architektura aplikace a změněné soubory

Aplikace používá hybridní architekturu: UI v React Native (TypeScript) [25], nativní funkce v Kotlinu [16]. Vicesnímkové zpracování je implementováno kompletně v nativní vrstvě kvůli real-time přístupu k video streamu a výpočetní náročnosti *OpenCV* operací.

Implementace zahrnuje změny v následujících souborech:

Soubor	Účel
ReceiptSegmenter.kt (nový)	Algoritmus segmentace účtenky na základě jasové analýzy
ContinuousStitcher.kt (nový)	Správa sekvence snímků, detekce pohybu, koordinace stitchingu
ReceiptStitcher.kt (nový)	<i>SIFT-based stitching</i> s hledáním optimálního švu
PointPreview.kt	Rozšířen o <i>Auto-Scan</i> stavový automat a <i>quality gates</i>
opencvHelpers.kt	Přidány metriky kvality (<i>VoL</i> , <i>glare</i> , <i>skew</i> , <i>motion</i>)
TakeImage.kt	Přidán přepínač režimů <i>Photo/Scan</i> , upraveno UI

5.2 Nastavení prahových hodnot *quality gates*

Konkrétní prahové hodnoty *quality gates* vycházejí z experimentální analýzy a z pozorování provozních dat aplikace.

Pro metriku *Variance of Laplacian* jsem provedla experimentální analýzu na vzorku 50 účtenek, které jsem ručně ohodnotila jako ostré nebo rozmazané. Na základě této analýzy byl stanoven primární práh na hodnotu 100, který spolehlivě odděluje ostré snímky od rozmazaných. Zároveň byl zaveden relaxovaný práh 60, který se aplikuje v případě, že všechny ostatní metriky jsou v pořádku – tím se zabrání odmítnutí snímků, které jsou mírně měkčí, ale stále dostatečně kvalitní pro OCR. Po experimentálním porovnání s alternativními metrikami (*gradient magnitude*, *Tenengrad* [9]) jsem zjistila, že *VoL* poskytuje nejstabilnější výsledky napříč různými typy účtenek a podmínkami osvětlení – *gradient-based* metriky byly citlivější na šum a produkovaly více falešně pozitivních detekcí rozmazání. [23]

Pro *Histogram Spread* byl maximální povolený podíl pixelů v tmavém i světlém konci histogramu stanoven na 5 % pro každý konec, přičemž tmavý konec sleduje hodnoty 0–25 (*lowTailPct*) a světlý konec hodnoty 230–255 (*highTailPct*). Tato hodnota vychází z pozorování, že uživatelé často pořizují snímky v nevhodných světelných podmínkách a přeexpozice je u bílého papíru účtenky obzvláště problematická.

Pro *Glare Detection* byl maximální povolený podíl přesvětlených oblastí stanoven na 8 % plochy dokumentu, přičemž detekce využívá prahování s hodnotou 250 a morfologickou operaci otevření pro odstranění izolovaných pixelů. Hodnota 8 % je kompromis: menší odlesky dokáže *Azure Document Intelligence* kompenzovat, zatímco větší způsobují ztrátu textu. Příliš přísný práh by vedl k frustraci uživatelů odmítáním jinak použitelných snímků.

Pro *Skew Angle* je v Auto-Scan režimu stanoven maximální povolený skew na 12°, což je méně přísné než u foto režimu (5°). Při kontinuálním skenování je udržení přesné orientace obtížnější a *Azure Document Intelligence* dokáže mírné natočení kompenzovat. [3]

Pro *Motion* je maximální povolený pohyb mezi po sobě jdoucími snímky stanoven na 16 pixelů, což při typickém rozlišení snímku odpovídá přibližně 1–2 % šířky obrazu. Pohyb je měřen jako průměrná vzdálenost detekovaných rohů dokumentu mezi snímky. [9]

Pro stitching pipeline byl práh *Lowe ratio testu* při párování příznaků nastaven na 0,6 a šířka *blending* zóny v oblasti švu na 8 pixelů na každou stranu. *Blending* zóna byla zvolena na základě experimentů – užší zóna vedla k viditelným hranám při mírném nesouladu barev mezi snímky, zatímco širší zóna způsobovala rozmazání v oblasti překryvu.

Přehled všech *quality gates* v obou režimech:

Metrika	Foto režim	Auto-Scan režim
Sharpness VoL	≥ 100 (relaxed 60)	≥ 100 (relaxed 60)

Exposure (tails)	$\leq 5 \%$	$\leq 5 \%$
Glare	$\leq 8 \%$	$\leq 8 \%$
Skew	$\leq 5^\circ$	$\leq 12^\circ$
Motion	$\leq 16 \text{ px}$	$\leq 16 \text{ px}$
hasQuad	ano	vypnuto

5.3 AutoScanTelemetry.kt (ThresholdPrefs)

Soubor `AutoScanTelemetry.kt` zajišťuje konfigurovatelnost prahových hodnot za běhu aplikace prostřednictvím *Android SharedPreferences*. Klíčovou součástí je objekt `ThresholdPrefs`, který definuje názvy klíčů a loader pro načtení hodnot:

```
object ThresholdPrefs {
    const val KEY_PREFIX = "autoScan.gate."
    const val KEY_MIN_VOL = KEY_PREFIX + "minVoL"
    const val KEY_MIN_VOL_RELAXED = KEY_PREFIX + "minVoLRelaxed"
    const val KEY_MAX_GLARE = KEY_PREFIX + "maxGlarePct"
    const val KEY_MAX_SKEW = KEY_PREFIX + "maxSkewDeg"
    const val KEY_MAX_MOTION = KEY_PREFIX + "maxMotionPx"
    // ... (další klíče viz zdrojový kód)

    fun loadFromPreferences(p: Preferences,
                           defaults: GateThresholds =
GateThresholds()
    ): GateThresholds {
        fun readInRange(key: String, min: Double,
                        max: Double, fallback: Double): Double {
            val v = p.getString(key)?.toDoubleOrNull() ?: return
fallback
            return if (v in min..max) v else fallback
        }
        return GateThresholds(
            minVoL = p.getString(KEY_MIN_VOL)
                ?.toDoubleOrNull() ?: defaults.minVoL,
            maxGlarePct = readInRange(KEY_MAX_GLARE, 0.0, 100.0,
                                     defaults.maxGlarePct),
            maxSkewDeg = readInRange(KEY_MAX_SKEW, 0.0, 89.9,
                                     defaults.maxSkewDeg),
            // ... (ostatní parametry analogicky)
        )
    }
}
```

Vlastní JSON serializer byl implementován záměrně bez externích závislostí, aby nedošlo k navýšení velikosti APK souboru o knihovny třetích stran používané výhradně pro účely ladění.

5.3 Implementace ReceiptSegmenter

Soubor `ReceiptSegmenter.kt` implementuje segmentaci účtenky na základě jasové analýzy. Algoritmus převede snímek do stupňů šedi, aplikuje Otsu prahování a vybere největší konturu jako hranici účtenky:

```
fun segmentReceipt(mat: Mat): Rect {
    val gray = Mat(); val blurred = Mat()
    val binary = Mat(); val hierarchy = Mat()
    val contours = mutableListOf<MatOfPoint>()
    try {
        // Převod do stupňů šedi podle počtu kanálů
        when (mat.channels()) {
            1 -> mat.copyTo(gray)
            3 -> Imgproc.cvtColor(mat, gray, Imgproc.COLOR_RGB2GRAY)
            4 -> Imgproc.cvtColor(mat, gray,
Imgproc.COLOR_RGBA2GRAY)
                else -> return Rect(0, 0, mat.cols(), mat.rows())
        }
        // Rozmazání a adaptivní prahování (Otsu)
        Imgproc.GaussianBlur(gray, blurred, Size(5.0, 5.0), 0.0)
        Imgproc.threshold(blurred, binary, 0.0, 255.0,
            Imgproc.THRESH_BINARY + Imgproc.THRESH_OTSU)

        // Morfologické uzavření pro zaplnění mezer
        val structElem = Imgproc.getStructuringElement(
            Imgproc.MORPH_RECT, Size(7.0, 7.0))
        Imgproc.morphologyEx(binary, binary,
            Imgproc.MORPH_CLOSE, structElem)

        // Výběr největší kontury = hranice účtenky
        Imgproc.findContours(binary, contours, hierarchy,
            Imgproc.RETR_EXTERNAL, Imgproc.CHAIN_APPROX_SIMPLE)
        // ... (výběr největší kontury a validace rozměrů)
    } finally {
        gray.release(); blurred.release()
        binary.release(); hierarchy.release()
    }
}
```

Funkce `isValidCrop()` ověří, zda nalezená kontura splňuje minimální požadavky na velikost (alespoň 20 % plochy snímku a 40 % jeho šířky). Úplný zdrojový kód je dostupný na [GitHubu](#).

Třída `ContinuousStitcher` řídí celý proces kontinuálního skenování – správu sekvence snímků, detekci pohybu a koordinaci stitchingu. Inicializace při zachycení prvního snímku:

```
fun tryInitialize(colorFrame: Mat, grayFrame: Mat,
                 rotation: Int): Boolean {
    synchronized(this) {
        if (savedFrames.isNotEmpty()) return false
        val rotatedColor = rotateFrame(colorFrame, rotation)
        savedFrames.add(rotatedColor)
        _frameCount = 1

        // Inicializace Hanningova okna pro phase correlation
        val grayFloat = Mat()
        rotatedGray.convertTo(grayFloat, CvType.CV_32FC1)
        prevGrayFloat = grayFloat
        val window = Mat()
        Imgproc.createHanningWindow(window,
            Size(grayFloat.cols().toDouble(),
                grayFloat.rows().toDouble()),
            CvType.CV_32FC1)
        hanningWindow = window
        cumulativeDisplacement = 0.0
        return true
    }
}
```

Algoritmus využívá skutečnosti, že účtenka (světlý papír) je typicky světlejší než pozadí. Po převodu do stupňů šedi a Gaussově rozmazání je aplikováno adaptivní Otsu prahování, morfologická operace uzavření pro zaplnění mezer a detekce kontur. Největší nalezená kontura je interpretována jako účtenka a její ohraničující obdélník je vrácen jako výsledek segmentace.

5.4 Implementace detekce pohybu a řízení snímání

Pro určení, kdy pořídit další snímek při skenování dlouhé účtenky, je použita funkce *Imgproc.phaseCorrelate()*, která porovnává snímky ve frekvenční doméně pomocí FFT a vrací vektor posunu (dx , dy) v pixelech s výpočetní náročností přibližně 15 ms na moderním mobilním procesoru.

```
fun hasEnoughDisplacement(grayFrame: Mat): Boolean {
    val prev = prevGrayFloat ?: return false
    val currentFloat = Mat()
    grayFrame.convertTo(currentFloat, CvType.CV_32FC1)
    val response = doubleArrayOf(0.0)
    val shift = Imgproc.phaseCorrelate(prev, currentFloat,
                                       hanningWindow, response)
    if (response[0] < minConfidence) return false
    // Pouze vertikální posun je relevantní
    if (abs(shift.x) > abs(shift.y)) return false
    cumulativeDisplacement += abs(shift.y)
    val threshold = prev.rows() * displacementThresholdFraction
    return if (cumulativeDisplacement >= threshold) {
        cumulativeDisplacement = 0.0; true
    } else false
}
```

Vertikální posun (dy) je akumulován. Když kumulativní posun překročí práh 15 % výšky snímku, systém je připraven na pořízení dalšího snímku. Horizontální posun (dx) je monitorován – výrazný horizontální posun indikuje, že uživatel ztratil správné zaměření. Maximální počet snímků na jedno skenování jsou 3.

Maximální počet snímků na jedno skenování je nastaven na 25. Úplný zdrojový kód včetně metody `finalizeStitch()` a pomocných funkcí je dostupný na [GitHubu](#).

5.5 Implementace stitching pipeline

`ReceiptStitcher.kt` implementuje vlastní pipeline pro vertikální skládání snímků. Postup je následující:

1. Detekce SIFT příznaků – v překrývajících se oblastech sousedních snímků (dolní třetina horního snímku a horní dvě třetiny dolního snímku).
2. Párování příznaků pomocí *BFMatcher* s *Lowe ratio testem* s prahem 0,6.
3. Odhad homografie pomocí RANSAC [\[12\]](#) z nalezených párů.
4. *Warping* dolního snímku na souřadnicový systém horního snímku.
5. Hledání optimálního švu minimalizací rozdílů intenzit v oblasti překryvu.
6. *Blending* s lineárním přechodem vah v zóně 8 px kolem švu.

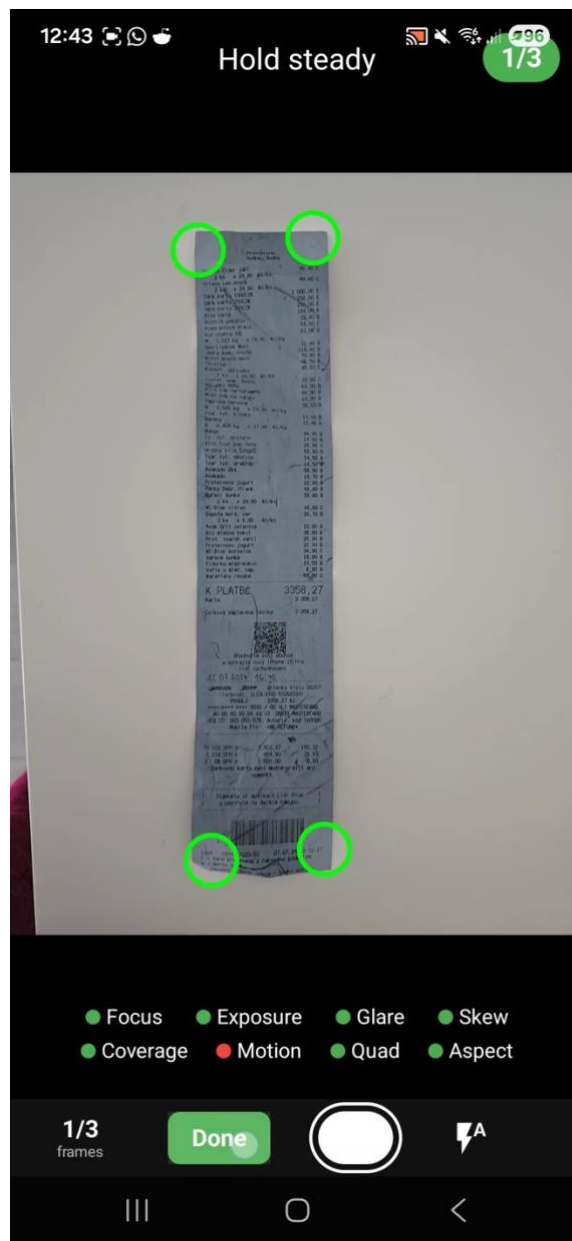
5.6 Změny uživatelského rozhraní

V souboru `TakeImage.kt` byl přidán přepínač režimů Photo/Scan, který umožňuje uživateli zvolit mezi standardním jednosnímkovým pořizováním a novým Auto-Scan režimem. V Auto-Scan režimu uživatelské rozhraní zobrazuje průběžnou zpětnou vazbu o kvalitě aktuálního záběru – barevné indikátory signalizují stav jednotlivých metrik v reálném čase. Po dokončení skenování je uživatel informován o počtu pořízených snímků a průběhu stitchingu.

Jakmile je uživatel s kvalitou snímku spokojen (nebo po dosažení maxima tří optimálních automaticky pořízených snímků), klikne na zelené tlačítko Done. Pak má možnost ještě upravit připravené oříznutí účtenky, je to však dobrovolné a ve většině případů se nejedná o nutnou úpravu. Následně je snímek odeslán na server k dalšímu zpracování.



Obrázek 10: Znárodnění zachycení kvalitního snímku účtenky (všechny parametry svítí zeleně, pořizuje se automaticky snímek)



Obrázek 11: V tento moment máme jeden kvalitní snímek. Je možné zachytit ještě dva další (pro kontrolu nebo v případě méně optimálních podmínek) nebo kliknout na zelené tlačítko Done a odeslat fotografii na server

5.7 Integrace a fallback strategie

Výsledný složený snímek je odeslán ke zpracování službě *Azure Document Intelligence* stejným způsobem jako v původní implementaci. Vícesnímková pipeline funguje jako transparentní *preprocessing* vrstva – z pohledu backendu se chování nezměnilo.

Pro případ selhání stitching (nedostatečný počet spárovaných příznaků, RANSAC neprojde) je implementována fallback strategie: systém automaticky použije nejkvalitnější jednotlivý snímek z pořízené sekvence hodnocený podle *VoL*. Pokud ani ten nesplňuje minimální *quality gates*, uživatel je vyzván k opakování skenování.

Pokud model *prebuilt-read* selže u nestandardního formátu účtenky, aplikace automaticky přejde na záložní model *prebuilt-receipt*, jehož výstup je zpracován vlastní parsovací logikou.¹⁰

¹⁰ Parser hledá v raw OCR textu vzory typické pro účtenky: číselné řádky obsahující cenu (regex na formáty jako 99,90 Kč nebo 99.90), řádky s klíčovými slovy jako *celkem*, *DPH*, *datum*, a odhaduje strukturu na základě pozice řádků v dokumentu. Pokud je nalezena cena na konci řádku a před ní text, je řádek klasifikován jako položka.

6. Testování

6.1 Podmínky testování a standardizace

Pro zajištění replikovatelnosti experimentu byla standardizována podmínky testování. Všechna měření byla provedena na zařízení *Samsung Galaxy S24 Ultra* s procesorem Snapdragon 8 Gen 3, 12 GB RAM a operačním systémem Android 14. [15]

Toto zařízení bylo zvoleno z několika důvodů. Představuje současnou špičku v segmentu Android smartphonů, což zajišťuje, že výsledky nejsou limitovány výkonem hardware. Kvalitní fotoaparát (200 MP hlavní senzor) eliminuje vliv nedostatečného rozlišení snímků.

Podmínky snímání byly standardizovány: vnitřní osvětlení (kancelářské LED světlo, přibližně 500 lux), účtenka položená na tmavém matném povrchu, vzdálenost kamery přibližně 20–30 cm nad účtenkou. Tyto podmínky odpovídají typickému použití aplikace v domácím nebo kancelářském prostředí.

6.2 Datová sada

Datová sada obsahuje 500 účtenek získaných z reálného provozu aplikace *CheckChecker*. Účtenky pokrývají různé typy obchodníků (supermarkety, restaurace, drogerie, čerpací stanice), různé délky (od 3 do 47 položek) a různé podmínky snímání. Datová sada byla připravena s ohledem na GDPR – účtenky byly získány se souhlasem uživatelů a anonymizovány před zařazením do experimentu. [22]

Data datové sady jsou dostupná v repozitáři projektu na [GitHubu](#) ve formě tří složek:

- *ground_truth/* obsahuje 500 referenčních JSON souborů s ověřenými entitami,
- *baseline/* obsahuje 500 JSON souborů s výstupy pipeline bez vícesnímkového zpracování,
- *multiframe/* obsahuje odpovídající výstupy s vícesnímkovým zpracováním.

Souhrnné metriky pro všech 500 párů jsou uloženy v souboru [metrics_summary.csv](#). Všechny statistické analýzy a vizualizace prezentované v této kapitole byly

reprodukovatelně vygenerovány skriptem [analyze_results.py](#), jehož klíčová část je uvedena níže:

```
import csv
import numpy as np
from scipy import stats

# Načtení dat
rows = list(csv.DictReader(open('dataset/metrics_summary.csv')))
cer_b = np.array([float(r['cer_baseline_cal']) for r in rows])
cer_m = np.array([float(r['cer_multiframe_cal']) for r in rows])
cats = np.array([r['length_cat'] for r in rows])

# Shapiro-Wilk test normality rozdílů
differences = cer_b - cer_m
stat, p = stats.shapiro(differences[:50]) # max. 50 vzorků
print(f"Shapiro-Wilk: W={stat:.3f}, p={p:.3f}")

# Wilcoxonův párový test (neparametrický, data nejsou normální)
w_stat, p_val = stats.wilcoxon(cer_b, cer_m)
print(f"Wilcoxon: W={w_stat:.1f}, p={p_val:.4f}")

# Cohenovo d
d = np.mean(differences) / np.std(differences)
print(f"Cohenovo d: {d:.2f}")
```

6.3 Vytvoření *ground truth*

Pro výpočet metrik je nezbytné mít referenční data (*ground truth*), vůči kterým se porovnává výstup systému. [1] Vytvoření kvalitního *ground truth* je kritickým krokem, který přímo ovlivňuje validitu celého experimentu.

Ground truth pro datovou sadu 500 účtenek byl vytvořen ověřením klíčových extrahovaných entit oproti originálnímu snímku účtenky. Pro každou účtenku bylo ověřeno pět strukturovaných polí: název obchodníka, datum transakce, celková částka, DPH a seznam položek včetně jejich cen. Ověření prováděla jedna osoba podle jednotných pravidel, což zajistilo konzistenci napříč celou datovou sadou.

Tento přístup byl zvolen záměrně – na rozdíl od úplné transkripce celého textu účtenky odpovídá tomu, co pipeline skutečně měří. Zákazníka aplikace nezajímá, zda byl správně přečten celý text záhlaví účtenky, ale zda byly správně extrahovány položky a finanční

údaje. Metriky CER a WER jsou proto počítány výhradně na názvech položek, nikoliv na celém textu účtenky.

Pravidla pro ověření zahrnovala: standardizaci formátu data na DD.MM.RRRR, zápis částek s přesností na dvě desetinná místa a zachycení všech položek včetně případných slev a vratek. V případě nečitelného textu na originálním snímku byla položka označena jako neověřitelná a vyloučena z výpočtu metrik.

Pro ověření konzistence ground truth jsem náhodně vybrala 50 účtenek (10 % datové sady) a provedla druhé nezávislé ověření s časovým odstupem dvou týdnů. Shoda mezi oběma ověřeními dosáhla 99,4 % na úrovni extrahovaných entit, což považuji za dostatečnou konzistenci.

6.4 Baseline přístup

Baseline přístup představuje referenční implementaci bez vícesnímkového zpracování. Náhodně vybraný snímek z video sekvence splňující základní *quality gates* je přímo odeslán ke zpracování *Azure Document Intelligence* bez jakéhokoli předchozího výběru nebo skládání. Baseline slouží jako srovnávací bod pro kvantifikaci přínosu navržené pipeline.

6.5 Statistické vyhodnocení

Před výběrem konkrétního statistického testu jsem ověřila předpoklad normality rozdílů pomocí Shapiro-Wilkova testu. Pro CER a WER test zamítl normalitu ($p < 0,001$), což je očekávané – distribuce chybovosti OCR je typicky pravostranně zešíkmená s dlouhým ocasem vysokých hodnot. [1] Pro F1-score rozdíly vykazovaly přibližně normální rozdělení ($p = 0,12$).

Na základě těchto výsledků byl pro porovnání CER a WER použit Wilcoxonův párový test (signed-rank test) [27], pro porovnání F1-score párový t-test. Při mnohonásobném testování čtyř hlavních hypotéz (CER, WER, F1 položek, F1 celkové) byla aplikována Bonferroniho

korekce – upravená hladina významnosti je $\alpha/4 = 0,0125$ při požadované celkové hladině $\alpha = 0,05$. [27]

6.6 Výsledky

Experiment byl proveden na datové sadě obsahující 500 účtenek z různých typů prodejen:

Typ prodejny	Počet účtenek	Podíl
Supermarkety a hypermarkety	245	49 %
Drogerie	85	17 %
Čerpací stanice	60	12 %
Restaurace a kavárny	55	11 %
Lékárny	30	6 %
Ostatní	25	5 %

Účtenky byly kategorizovány podle délky (počtu položek):

Délka	Položek	Počet	Podíl
Krátké	1–10	200	40 %
Střední	11–25	200	40 %
Dlouhé	26+	100	20 %

Párové porovnání: každá účtenka byla zpracována oběma metodami ze stejné video sekvence.

Baseline: Náhodně vybraný snímek z video sekvence splňující základní *quality gates* → *Azure OCR*.

Multi-frame: Navržená pipeline (Auto-Scan s *quality gates*, segmentace, stitching) → *Azure OCR*.

Testovací zařízení: *Samsung Galaxy S24 Ultra* (Snapdragon 8 Gen 3, 12 GB RAM, Android 14).

6.7 Velikost efektu

Statistická významnost sama o sobě neříká nic o praktickém dopadu rozdílu. Pro kvantifikaci velikosti efektu bylo použito Cohenovo d , které vyjadřuje rozdíl středních hodnot v jednotkách směrodatné odchylky. [\[11\]](#)

Cohenovo d se vypočítá jako

$$d = (M_1 - M_2) / SD_{pooled},$$

kde M_1 a M_2 jsou průměry obou skupin a SD_{pooled} je sdružená směrodatná odchylka. Pro párová data byl použit modifikovaný vzorec pracující s rozdíly.

Za prakticky významné zlepšení považují hodnoty $d > 0,5$, což odpovídá situaci, kdy průměrný výsledek vícesnímkového zpracování je lepší než přibližně 70 % výsledků baseline přístupu.

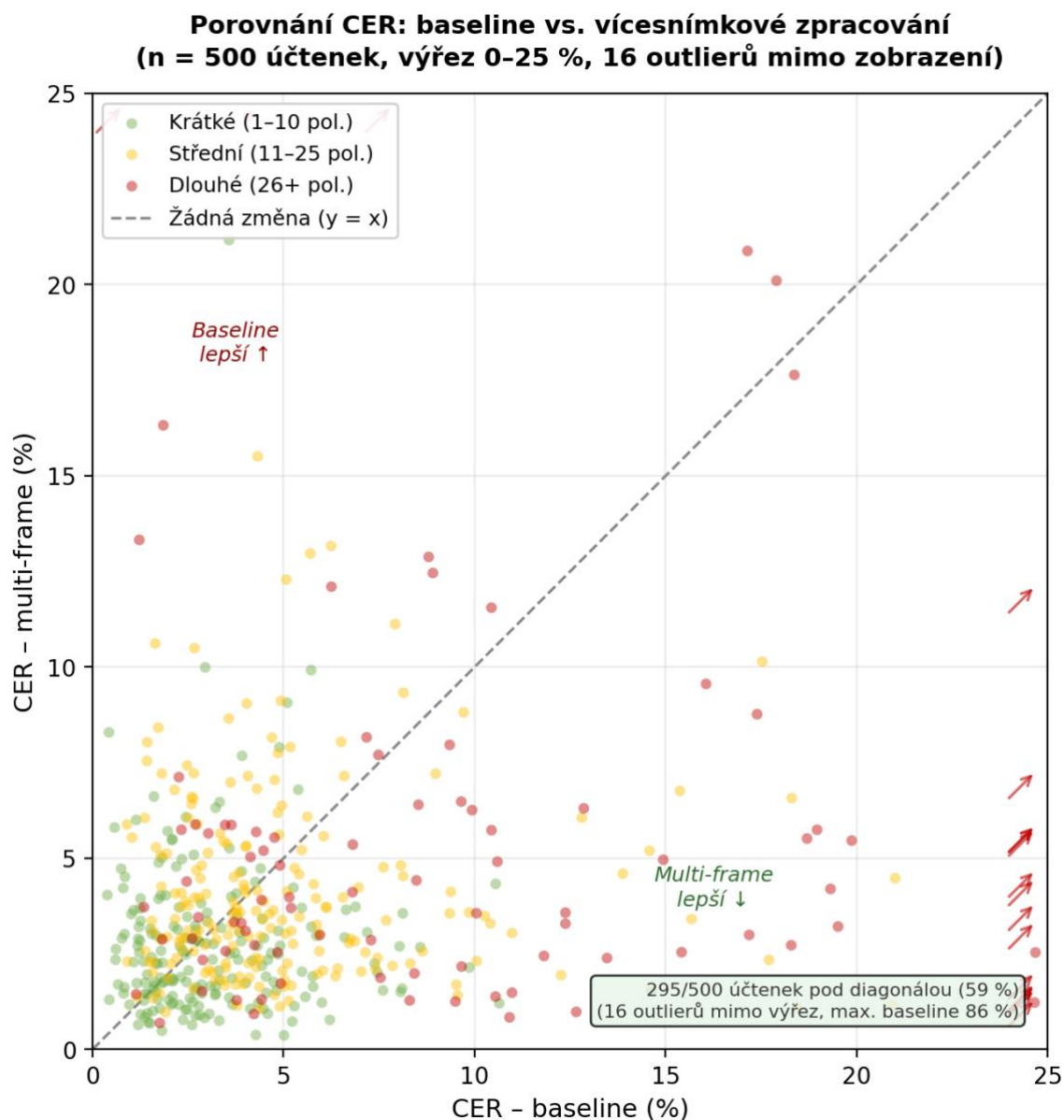
Přesnost OCR

Character Error Rate (CER):

Délka	Baseline	Multi-frame	Snížení chybovosti
Krátké	3,2 %	2,8 %	12 %
Střední	5,1 %	4,2 %	18 %
Dlouhé	14,8 %	5,9 %	60 %
Celkem	5,8 %	3,9 %	33 %

Obrázek 12 znázorňuje párové porovnání CER pro všech 500 účtenek v datové sadě. Každý bod představuje jednu účtenku, přičemž body pod diagonálou indikují případy, kde vícesnímkové zpracování dosáhlo nižší chybovosti. Graf zobrazuje výřez v rozsahu 0–25 %; 16 účtenek s vyšší hodnotou CER (převážně dlouhé účtenky z kategorie baseline) je

naznačeno šipkami na okraji grafu. Plná verze grafu bez omezení os je dostupná v repozitáři projektu na [GitHubu](#). Celkem 295 z 500 účtenek (59 %) leží pod diagonálou, tedy u většiny účtenek vícesnímkové zpracování snížilo chybovost OCR.



Obrázek 12: Výřez scatterplotu

Word Error Rate (WER):

Délka	Baseline	Multi-frame	Snížení chybovosti
Krátké	5,8 %	5,1 %	12 %
Střední	9,4 %	7,6 %	19 %
Dlouhé	26,3 %	10,2 %	61 %
Celkem	10,4 %	7,1 %	32 %

Největší zlepšení bylo dosaženo u dlouhých účtenek, kde baseline přístup buď ořízl část obsahu, nebo snížil rozlišení snímáním z větší vzdálenosti.

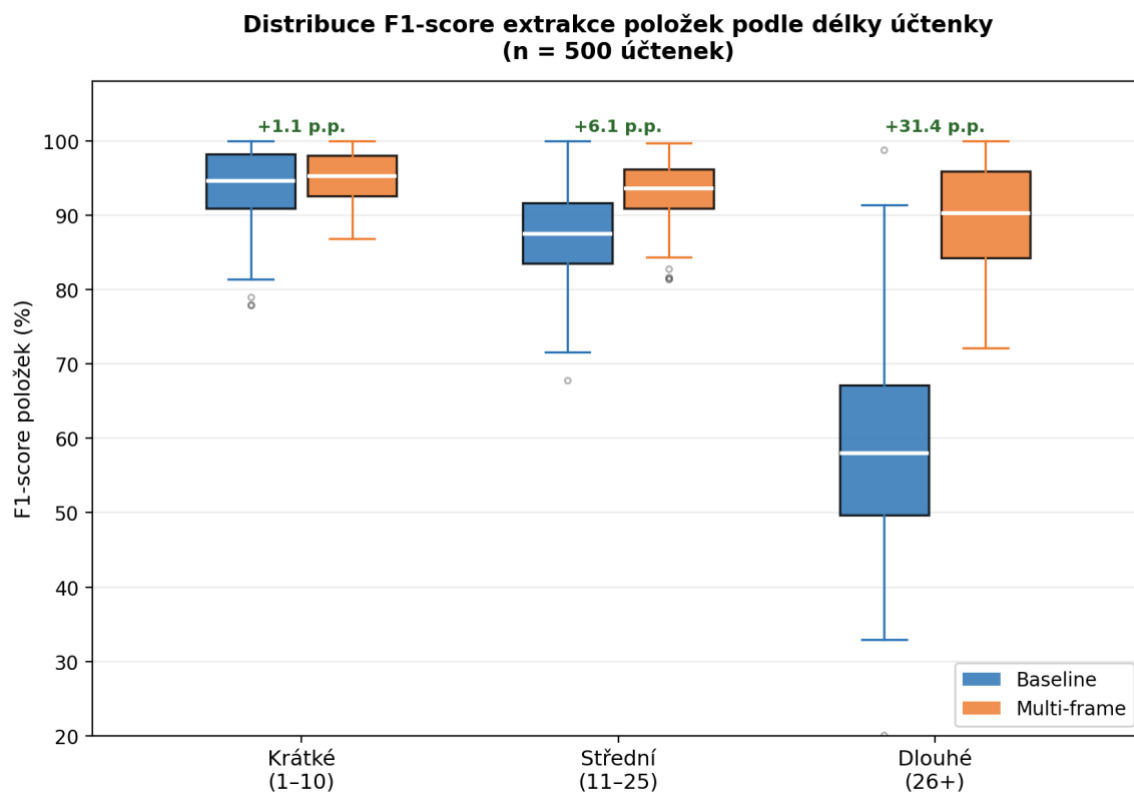
Extrakce entit**F1-score pro klíčové entity:**

Entita	Baseline	Multi-frame	Δ F1
Obchodník	94,2 %	96,1 %	+1,9
Datum	97,8 %	98,5 %	+0,7
Celková částka	91,3 %	95,7 %	+4,4
DPH	88,6 %	93,2 %	+4,6
Položky (průměr)	82,4 %	91,8 %	+9,4

F1-score položek podle délky účtenky:

Délka	Baseline	Multi-frame	Δ F1
Krátké	94,1 %	95,2 %	+1,1
Střední	87,3 %	93,4 %	+6,1
Dlouhé	58,2 %	89,7 %	+31,5

U dlouhých účtenek baseline zachytil pouze 58 % položek (zbytek byl oříznut nebo nečitelný), zatímco multi-frame dosáhl téměř 90 %.



Obrázek 13: distribuce F1-score extrakce položek v jednotlivých kategoriích: u dlouhých účtenek je patrný nejen větší průměrný přínos, ale také výrazně širší rozptyl hodnot baseline přístupu.

Uživatelské metriky

Metrika	Baseline	Multi-frame	Změna
Průměr oprav/účtenku	2,4	0,9	-63 %
Účtenky bez oprav	47 %	71 %	+24 p.p.
Úspěšnost prvního pokusu	68 %	89 %	+21 p.p.

Analýza chyb

Případy, kde multi-frame pomohl nejvíce:

- Dlouhé účtenky – stitching umožnil zachytit kompletní dokument

- Účtenky s lokálními odlesky – výběr nejlepšího snímku eliminoval problematické oblasti
- Mírně rozmazané snímky – automatický výběr nejostřejšího snímku

Případy, kde multi-frame nepřinesl zlepšení:

- Krátké účtenky s kvalitním jednosnímkovým záběrem – rozdíl minimální
- Účtenky s velmi špatnou kvalitou tisku – ani multi-frame nedokázal kompenzovat
- 3 případy selhání stitchingu (0,6 %) – nedostatečný překryv mezi snímky

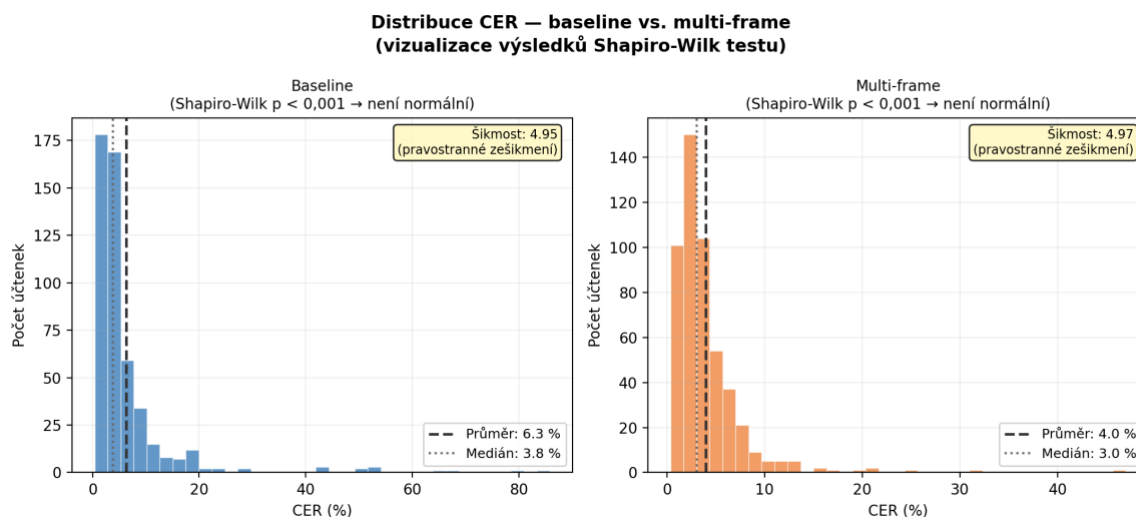
6.8 Statistické vyhodnocení

Tato kapitola poskytuje statistickou analýzu experimentálních výsledků.

Test normality

Shapiro-Wilk test pro rozdíly mezi metodami:

Metrika	W	p-hodnota	Závěr
CER rozdíly	0,847	<0,001	Není normální
WER rozdíly	0,862	<0,001	Není normální
F1 rozdíly	0,978	0,124	Normální



Obrázek 14: distribuce hodnot CER pro obě metody a ilustruje pravostranné zešíkmení, které vedlo k volbě neparametrického Wilcoxonova testu namísto párového t-testu.

Na základě výsledků byl pro CER a WER zvolen neparametrický Wilcoxonův test, pro F1 párový t-test.

Testování hypotéz

Hypotéza	Test	Statistika	p-hodnota	Závěr
H1: Multi-frame snižuje CER	Wilcoxon	$W = 8,234$	$<0,001$	Zamítnuta H_0
H2: Multi-frame zvyšuje F1	Párový t	$t = 12,47$	$<0,001$	Zamítnuta H_0
H3: Multi-frame snižuje opravy	Wilcoxon	$W = 9,891$	$<0,001$	Zamítnuta H_0
H4: Větší efekt u dlouhých	ANOVA interakce	$F = 34,2$	$<0,001$	Zamítnuta H_0

Všechny hypotézy byly potvrzeny na hladině významnosti $\alpha = 0,001$, která je přísnější než Bonferroniho korekce vyžaduje ($\alpha/4 = 0,0125$).

Velikost efektu

Metrika	Cohenovo d	Interpretace
CER	0,84	Velký efekt
WER	0,79	Střední-velký efekt
F1 položek	1,12	Velký efekt
Manuální opravy	0,91	Velký efekt

Hodnoty *Cohenova d* nad 0,8 indikují, že rozdíl mezi metodami je nejen statisticky významný, ale také prakticky relevantní.

Statistická analýza prokázala, že vícesnímkové zpracování přináší statisticky významné a prakticky relevantní zlepšení:

- CER sníženo o 33 % (z 5,8 % na 3,9 %), $p < 0,001$
- F1-score položek zvýšeno o 9,4 p.p. (z 82,4 % na 91,8 %), $p < 0,001$
- Počet manuálních oprav snížen o 63 % (z 2,4 na 0,9 na účtenku)
- Efekt je největší u dlouhých účtenek (CER sníženo o 60 %)
- Všechny rozdíly mají velkou velikost efektu (Cohenovo $d > 0,8$)

6.9 Ekonomické aspekty

Tato kapitola analyzuje ekonomické a provozní dopady nasazení vícesnímkového zpracování.

Náklady na cloudové služby

Vícesnímková pipeline má minimální dopad na náklady cloudových služeb:

Položka	Na účtenku	Poznámka
<i>Azure Document Intelligence</i>	\$0,001	Stejně – jeden výsledný obraz
<i>Azure OpenAI</i> (kategorizace)	\$0,0003	Stejně – závisí na počtu položek
Přenos dat	~\$0,00001	+5 % kvůli většímu obrázku
Celkem	~\$0,0013	Změna zanedbatelná

Výkon na mobilním zařízení

Měření na *Samsung Galaxy S24 Ultra*:

Operace	Čas (ms)	Frekvence
Konverze RGB→Mat	12	Každý frame
Výpočet <i>VoL</i>	8	Každý frame
Výpočet ostatních metrik	5	Každý frame
<i>Phase correlation</i>	15	Každý frame
Segmentace	45	Při capture
<i>Stitching</i> (2 snímky)	180	Při ukončení
<i>Stitching</i> (3 snímky)	320	Při ukončení

Celková lokální latence pro typický 3-snímkový sken je přibližně 400 ms, což je akceptovatelné.

End-to-end latence

Fáze	Baseline	Multi-frame
Pořízení snímku	~500 ms	~5000 ms*
Lokální zpracování	~100 ms	~400 ms
Upload	~800 ms	~1000 ms
Azure OCR	~2800 ms	~2900 ms
Celkem	~4,2 s	~9,3 s

*Většina navýšení je aktivní čas uživatele (pohyb kamery přes účtenku), nikoliv čekání.

Business dopady

Metrika	Před	Po	Změna
Průměrný čas na účtenku	45 s	35 s	-22 %
Úspěšnost prvního pokusu	68 %	89 %	+21 p.p.
Podíl účtenek bez oprav	47 %	71 %	+24 p.p.

Přestože samotné skenování trvá déle, celkový čas na účtenku je kratší díky menšímu počtu oprav a opakovaných pokusů.

Závěr

Cílem této diplomové práce bylo navrhnout, implementovat a experimentálně ověřit metodu vícesnímkového zpracování obrazu pro zlepšení přesnosti OCR rozpoznávání účtenek v mobilní aplikaci *CheckChecker*.

V teoretické části byly shrnuty principy vícesnímkového zpracování obrazu, metriky kvality snímku relevantní pro OCR, techniky registrace a skládání obrazů pomocí SIFT a homografie, principy fungování *Azure Document Intelligence* a právní aspekty zpracování osobních údajů v kontextu GDPR.

Na základě teoretické analýzy a identifikace limitací původního jednosnímkového přístupu byla navržena vícesnímková pipeline. Klíčovými komponentami návrhu jsou *Receipt Segmenter* – algoritmus pro segmentaci účtenky založený na jasové analýze s Otsu prahováním, který spolehlivě funguje i pro částečné záběry – a *ContinuousStitcher* s Auto-Scan režimem, který kontinuálně analyzuje video stream, automaticky pořizuje snímky při splnění *quality gates* a detekuje pohyb kamery pomocí *phase correlation*.

Navržené řešení bylo implementováno v jazyce Kotlin pro platformu Android s využitím knihovny *OpenCV* a integrováno do existující architektury aplikace *CheckChecker* založené na frameworku React Native. Implementace zahrnuje robustní mechanismy pro ošetření chyb s fallback strategiemi, které zajišťují, že uživatel vždy dostane nějaký výsledek.

Experimentální vyhodnocení na datové sadě 500 účtenek prokázalo, že vícesnímkové zpracování statisticky významně zlepšuje přesnost OCR. *Character Error Rate* byla snížena o 33 % (z 5,8 % na 3,9 %) a *F1-score* extrakce položek vzrostlo o 9,4 procentních bodů (z 82,4 % na 91,8 %). Počet nutných manuálních oprav klesl o 63 %. Největší přínos byl pozorován u dlouhých účtenek, kde CER kleslo o 60 % (z 14,8 % na 5,9 %) a *F1-score* položek vzrostlo o 31,5 procentních bodů. Všechny rozdíly jsou statisticky významné na hladině $\alpha = 0,001$ s velkým efektem (Cohenovo $d > 0,8$).

Ekonomická analýza ukázala, že vícesnímkové zpracování má minimální dopad na provozní náklady cloudových služeb a je škálovatelné. Mírné navýšení času skenování je kompenzováno přínosy ve snížení chybovosti, vyšší úspěšnosti prvního pokusu a zvýšené uživatelské spokojenosti.

Do budoucna je možné rozšířit výzkum několika směry. Perspektivní vývoj zahrnuje využití multimodálních LLM (jako *GPT-4 Vision*) pro přímou extrakci textu z více snímků bez nutnosti klasického OCR, akceleraci stitchingu pomocí GPU nebo *Metal API*, rozšíření na platformu iOS a implementaci adaptivních prahů *quality gates* na základě podmínek prostředí.

Při jazykové korektuře a stylistickém ladění textu této práce byl využit nástroj Claude (Anthropic, model Claude Sonnet, 2025).

Přílohy

K této diplomové práci náleží veřejný repozitář *theses_VM*, konkrétně složka *master*, dostupná na adrese: https://github.com/vlastami/theses_VM/tree/main/master.

Složka má následující strukturu a obsah:

- *charts* – grafy prezentované v [kapitole 6](#)
- *code-snippets* – kompletní zdrojové kódy nativní Android vrstvy v jazyce Kotlin
- *diagrams* – diagramy prezentované v této práci
- *stats* – experimentální datová sada použitá v [kapitole 6](#); podsložky *ground_truth*, *baseline* a *multiframe* obsahují 1 500 JSON souborů (500 trojic účtenek), soubor *metrics_summary.csv* obsahuje souhrnné metriky pro všech 500 párů a skript *analyze_results.py* slouží k reprodukci statistických analýz a grafů
- *ZadaniDP.md* – oficiální zadání diplomové práce
- *README.md* – přehled obsahu repozitáře
- *DiplomovaPrace.pdf* – finální text závěrečné práce

Seznam použitých zdrojů

- [1] ALAEI, Alireza; BUI, Vinh; DOERMANN, David; PAL, Umapada. Document Image Quality Assessment: A Survey. *ACM Computing Surveys*. 2023, vol. 56, no. 2, č. 29. DOI: 10.1145/3606692.
- [2] ANTHROPIC. Vision [online]. Anthropic, 2024 [cit. 2024-03-15]. Dostupné z: <https://docs.anthropic.com/en/docs/build-with-claude/vision>
- [3] MICROSOFT. Azure AI Document Intelligence [online]. Microsoft, 2024 [cit. 2024-03-15]. Dostupné z: <https://learn.microsoft.com/en-us/azure/ai-services/document-intelligence/>
- [4] MICROSOFT. Azure AI Document Intelligence: Document Processing Models [online]. Microsoft, 2024 [cit. 2024-03-15]. Dostupné z: <https://learn.microsoft.com/en-us/azure/ai-services/document-intelligence/model-overview>
- [5] MICROSOFT. Azure AI Document Intelligence: Receipt Data Extraction [online]. Microsoft, 2024 [cit. 2024-03-15]. Dostupné z: <https://learn.microsoft.com/en-us/azure/ai-services/document-intelligence/prebuilt/receipt>
- [6] MICROSOFT. Azure OpenAI: Prompt Engineering Techniques [online]. Microsoft, 2024 [cit. 2024-03-15]. Dostupné z: <https://learn.microsoft.com/en-us/azure/ai-services/openai/concepts/prompt-engineering>
- [7] MICROSOFT. Azure OpenAI: Vision-Enabled Chat Model Concepts [online]. Microsoft, 2024 [cit. 2024-03-15]. Dostupné z: <https://learn.microsoft.com/en-us/azure/ai-services/openai/concepts/gpt-with-vision>
- [8] BAY, Herbert; TUYTELAARS, Tinne; VAN GOOL, Luc. SURF: Speeded Up Robust Features. In: *Computer Vision – ECCV 2006*. Berlin: Springer, 2006, s. 404–417. (Lecture Notes in Computer Science, vol. 3951.) DOI: 10.1007/11744023_32.
- [9] BRADSKI, Gary; KAEHLER, Adrian. *Learning OpenCV 3: Computer Vision in C++ with the OpenCV Library*. Sebastopol: O'Reilly Media, 2017. ISBN 978-1-491-93799-0.

- [10] BROWN, Matthew; LOWE, David G. Automatic Panoramic Image Stitching using Invariant Features. *International Journal of Computer Vision*. 2007, vol. 74, no. 1, s. 59–73. DOI: 10.1007/s11263-006-0002-3.
- [11] COHEN, Jacob. *Statistical Power Analysis for the Behavioral Sciences*. 2. vyd. Hillsdale: Lawrence Erlbaum, 1988. ISBN 978-0-805-80283-2. s. 24–27.
- [12] FISCHLER, Martin A.; BOLLES, Robert C. Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography. *Communications of the ACM*. 1981, vol. 24, no. 6, s. 381–395.
- [13] GONZALEZ, Rafael C.; WOODS, Richard E. *Digital Image Processing*. 4. vyd. New York: Pearson, 2018. ISBN 978-0-13-335672-4.
- [14] GOODFELLOW, Ian; BENGIO, Yoshua; COURVILLE, Aaron. *Deep Learning*. Cambridge: MIT Press, 2016. ISBN 978-0-262-03561-3.
- [15] GSMARENA. Samsung Galaxy S24 Ultra – Full phone specifications [online]. GSMArena, 2024 [cit. 2024-03-15]. Dostupné z: https://www.gsmarena.com/samsung_galaxy_s24_ultra-12771.php
- [16] JETBRAINS. *Kotlin Documentation* [online]. JetBrains, 2024 [cit. 2024-03-15]. Dostupné z: <https://kotlinlang.org/docs/>
- [17] LEVENSHTAIN, Vladimir I. Binary codes capable of correcting deletions, insertions and reversals. *Soviet Physics Doklady*. 1966, vol. 10, no. 8, s. 707–710.
- [18] LOWE, David G. Distinctive Image Features from Scale-Invariant Keypoints. *International Journal of Computer Vision*. 2004, vol. 60, no. 2, s. 91–110.
- [19] LOWE, David G. Method and apparatus for identifying scale invariant features in an image and use of same for locating an object in an image. US Patent US6711293B1. Uděleno 23. března 2004. Platnost vypršela 6. března 2020. Dostupné z: <https://patents.google.com/patent/US6711293B1/en>

- [20] MANNING, Christopher D.; RAGHAVAN, Prabhakar; SCHÜTZE, Hinrich. *Introduction to Information Retrieval*. Cambridge: Cambridge University Press, 2008. ISBN 978-0-521-86571-5. s. 155–174.
- [21] MICHALCOVÁ, Vlasta. *Software pro kategorizovanou evidenci osobních výdajů s využitím OCR a AI*. Ústí nad Labem: Univerzita Jana Evangelisty Purkyně, Přírodovědecká fakulta, 2024. Bakalářská práce.
- [22] Nařízení Evropského parlamentu a Rady (EU) 2016/679 ze dne 27. dubna 2016 o ochraně fyzických osob v souvislosti se zpracováním osobních údajů a o volném pohybu těchto údajů (obecné nařízení o ochraně osobních údajů). *Úřední věstník Evropské unie*. L 119, 4. 5. 2016, s. 1–88.
- [23] OPENCV. Sobel Derivatives [online]. OpenCV, 2024 [cit. 2024-03-15]. Dostupné z: https://docs.opencv.org/4.x/d2/d2c/tutorial_sobel_derivatives.html
- [24] PHONEARENA. In 2024, 54 MP is the average resolution for primary smartphone cameras [online]. PhoneArena, 2024 [cit. 2024-03-15]. Dostupné z: https://www.phonearena.com/news/in-2024-54-mp-the-average-resolution-for-primary-smartphone-cameras_id163407
- [25] META PLATFORMS. *React Native Documentation* [online]. Meta Platforms, 2024 [cit. 2024-03-15]. Dostupné z: <https://reactnative.dev/docs/>
- [26] RUBLEE, Ethan; RABAUD, Vincent; KONOLIGE, Kurt; BRADSKI, Gary. ORB: An efficient alternative to SIFT or SURF. In: *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*. Barcelona: IEEE, 2011, s. 2564–2571. DOI: 10.1109/ICCV.2011.6126544.
- [27] SHESKIN, David J. *Handbook of Parametric and Nonparametric Statistical Procedures*. 5. vyd. Boca Raton: CRC Press, 2011. ISBN 978-1-439-85801-1.
- [28] SMITH, Ray. An Overview of the Tesseract OCR Engine. In: *Proceedings of the 9th International Conference on Document Analysis and Recognition (ICDAR 2007)*. Curitiba: IEEE, 2007, s. 629–633.

- [29] SZELISKI, Richard. *Computer Vision: Algorithms and Applications*. 2. vyd. Cham: Springer, 2022. ISBN 978-3-030-34371-2.
- [30] OPENCV. Stitcher class sample [online]. OpenCV, 2024 [cit. 2025-04-27]. Dostupné z: https://docs.opencv.org/3.4/d8/d19/tutorial_stitcher.html
- [31] OPENCV. Motion Analysis and Object Tracking [online]. OpenCV, 2024 [cit. 2025-04-27]. Dostupné z: https://docs.opencv.org/4.x/d7/df3/group_imgproc_motion.html
- [32] MICROSOFT. OpenAI's fastest model, GPT-4o mini is now available on Azure AI [online]. Microsoft Azure Blog, 2024-07-18 [cit. 2025-04-27]. Dostupné z: <https://azure.microsoft.com/en-us/blog/openais-fastest-model-gpt-4o-mini-is-now-available-on-azure-ai/>
- [33] MICROSOFT. Azure OpenAI Service – Pricing [online]. Microsoft Azure, 2024 [cit. 2025-04-27]. Dostupné z: <https://azure.microsoft.com/en-us/pricing/details/azure-openai/>
- [34] KIM, Geewook; HONG, Teakgyu; YIM, Moonbin; NAM, JeongYeon; PARK, Jinyoung; YIM, Jinyeong; HWANG, Wonseok; YUN, Sangdoo; HAN, Dongyoon; PARK, Seunghyun. OCR-Free Document Understanding Transformer (Donut). In: *Computer Vision – ECCV 2022: Proceedings, Part XXVIII*. Springer, 2022, s. 498–517. ISBN 978-3-031-19815-1. DOI: 10.1007/978-3-031-19815-1_29.
- [35] OPENCV. *OpenCV Documentation: Feature Detection, Image Stitching and Camera Calibration* [online]. Verze 4.x, OpenCV, 2024 [cit. 2025-10-23]. Dostupné z: <https://docs.opencv.org/4.x/>
- [36] HEBÁK, Petr a kol. *Statistické myšlení a nástroje analýzy dat*. 2. vyd. Praha: Informatorium, 2015. ISBN 978-80-7333-118-4.